Utils-Stack

tclutils · tkutils · ctrlutils · tical

2026-10-28

Inhaltsverzeichnis

1 Stack verstehen **4**
1.1 Einführung 4
1.2 Drei Repos, eine Schichtungsregel 4
1.3 Warum getrennt 4
1.4 Aufbau 4
1.5 Merkliste 5
2 Pairing und Setup **5**
2.1 Empfohlenes Pairing 5
2.2 Einmal Setup 6
2.3 In eine Tcl-Installation legen 7
2.4 Tests 9
2.5 Zusammenfassung 11
3 ctrlutils **11**
3.1 Fünf Controller 11
3.2 Abhängigkeit zur View 12
3.3 Weiter 13
3.4 Zusammenfassung 13
4 Filter: cufilter **13**
4.1 Merkliste 14
5 Verzeichnis beobachten: cumonitor **14**
5.1 Merkliste 14
6 Spiegeln: cusync **15**
6.1 compare 15
6.2 plan 16
6.3 apply und applyOne 16
6.4 Merkliste 16
7 cufileops und Undo **17**
7.1 install 17
7.2 Menü und Tasten 17
7.3 Öffnen 18
7.4 Kopieren, Ausschneiden, Einfügen 19
7.5 Mehrfachauswahl beim Rechtsklick 19
7.6 Sprache 20
7.7 cuundo 20
7.8 Alles zusammen 21
7.9 Zusammenfassung 22
8 Explorer und Dateiblick **22**
8.1 Zwei Rollen 22
8.2 Explorer (Kurz) 23
8.3 Dateiblick (Kurz) 24
8.4 Zusammenfassung 24
9 Checkliste App **24**
9.1 Zusammenfassung 25
10 Kalender: tical **25**
10.1 Was tical ist 25

10.2 Laden 26
10.3 Verhältnis zu tclutils und tkutils 26
10.4 Views 27
10.5 Termine 29
10.6 Renderer 30
10.6.1 Text: render::term 30
10.6.2 Tk: render::canvas 32
10.6.3 PDF: render::pdf 33
10.7 Feiertage 34
10.8 iCalendar 34
10.8.1 Ganztägige Termine 35
10.9 Import 35
10.10 Modell und SQLite 36
10.11 Beispiel 37
10.12 Fehler 38
10.13 Tests 39
10.14 Merkliste 39
11 Fallen **40**
11.1 tical 41
11.2 Behoben in ctrlutils 0.4 41
11.3 Behoben in ctrlutils 0.5 42
11.4 Zusammenfassung 42
12 Module (Stack-Sicht) **42**
12.1 tical 1.1.1 43
12.2 Zusammenfassung 44
13 Glossar **44**
13.1 Zusammenfassung 46
14 Fehlermeldungen **46**
14.1 Zusammenfassung 49
15 Lizenz **49**

1 Stack verstehen

1.1 Einführung

Der Utils-Stack besteht aus drei Bibliotheken: tclutils (Logik), tkutils (Widgets) und ctrlutils (Controller). Dieses Handbuch beschreibt, wie die drei zusammenpassen: Paarung der Versionen, Setup, Tests, die Controller-Module von ctrlutils und den Unterschied zwischen Explorer und Dateiblick. Ein eigenes Kapitel gilt der Kalender-Engine tical, einer vierten, optionalen Bibliothek, auf der zwei Kalender-Widgets von tkutils aufbauen. Die API der einzelnen Module beschreiben die Bücher *tclutils* und *tkutils*.

Es gilt für die Paarung tclutils **0.69.0**, tkutils **0.49.0** und ctrlutils **0.5**, dazu tical **1.1**. Beispiele und Testsuiten sind mit Tcl/Tk 8.6.14 und 9.0.4 unter Linux (Xvfb) geprüft, die Testsuiten auch unter macOS und Windows. Die Beispiele sind dort nicht geprüft.

1.2 Drei Repos, eine Schichtungsregel

```
tclutils   (tu*)   Logik, kein Tk        = Model
tkutils    (tku*)  Widgets               = View
ctrlutils  (cu*)   Verhalten verdrahten  = Controller

tical      (tical::*)  Kalender-Engine, eigenes Repo, optional
```

Ein tkutils-Modul **ist** ein Widget (`tkufilelist $path …`). Ein ctrlutils-Modul **verdrahtet** Verhalten auf vorhandenen Widgets (`cufileops::install …`) — ohne eigenen Widget-Pfad. Zwei Abweichungen: `cufilter::widget` baut eine eigene kleine Filterzeile, und `cusync` braucht gar kein Tk (Vergleich und Spiegeln über Provider).

Paketnamen folgen den Präfixen: Sammelpakete `tclutils`, `tkutils`, `ctrlutils`; Einzelmodule `tclutils::tu…`, `tkutils::tku…`, `ctrlutils::cu…`. tical gehört nicht zur Schichtung: Es hat einen eigenen Kern ohne Tk, eigene Renderer und wird über `auto_path` gefunden, nicht über den Modulpfad (Kapitel Kalender: tical).

1.3 Warum getrennt

- CI und Server laden nur tclutils.
- Widgets ohne App-Politik wiederverwenden.
- Controller gegen die Provider-API, nicht gegen eine feste App.

Tk-frei laden lassen sich `ctrlutils::cusync`, `ctrlutils::cumonitor` und `ctrlutils::cuundo`. Das Sammelpaket `ctrlutils` dagegen braucht Tk, denn `cufileops` zieht Kontextmenü und Dialoge aus tkutils nach. Geprüft (Tcl/Tk 8.6.14 und 9.0.4, Linux): ohne Display endet `package require ctrlutils` mit `no display name and no $DISPLAY environment variable`.

Paarung dieses Buchs: **0.69.0 / 0.49.0 / 0.4**, dazu tical **1.1**.

1.4 Aufbau

Kapitel 2 behandelt Paarung, Setup und Tests. Danach folgen die fünf Controller: Überblick, Filter, Verzeichnis beobachten, Spiegeln, `cufileops` und `cuundo`. Kapitel 8 stellt Explorer und

Dateiblick gegenüber, Kapitel 9 ist die Checkliste für eigene Anwendungen. Kapitel 10 beschreibt tical: Views, Termine, Renderer, Feiertage, iCalendar-Export und -Import und das Verhältnis zu den Kalendermodulen von tclutils und tkutils. Die Anhänge enthalten Fallen (A), die Modulliste (B), das Glossar (C) und Fehlermeldungen (D).

1.5 Merkliste

- Model / View / Controller als drei Paketbäume.
- Apps sind Glue darüber (Explorer, Dateiblick).
- Ohne Tk nur einzelne Controller laden, nicht das Sammelpaket.
- tical ist optional und liegt neben dem Stack, nicht darin.

Nächstes Kapitel: Pairing und Setup

2 Pairing und Setup

2.1 Empfohlenes Pairing

Lib	Version	braucht
tclutils	0.69.0	—
tkutils	0.49.0	tclutils 0.64 oder neuer (tusettings, tustr::graphemes, tudate::zones, tutestrun); 0.69.0 für den CSV-Export von tkutablelist und für tkusysinfo
ctrlutils	0.4	tkutils mit tkudialog 0.3 und tkufilelist 0.2
tical	1.1	nichts aus dem Stack; optional Tk, pdf4tcl, tdbc::sqlite3, für den Import tclutils (tuical, turrule)

Die CHANGELOGs von tclutils 0.69.0 und tkutils 0.49.0 nennen das Trio "tclutils 0.69.0 + tkutils 0.49.0 + ctrlutils 0.5". Der Umbrella von tkutils 0.49.0 braucht weiter nur tclutils 0.64.0 oder neuer; der CSV-Export von `tkutablelist` 0.5 und das Widget `tkusysinfo` brauchen 0.69.0. ctrlutils 0.5 braucht nichts Neues aus tclutils oder tkutils. Die Suiten unten sind mit genau diesem Trio gelaufen.

tical ist kein Teil des Trios. tclutils und ctrlutils laden es nie, tkutils nur in den beiden optionalen Widgets `tkutical` und `tkumonthcanvas`, die nicht im Sammelpaket stehen (Kapitel Kalender: tical).

Im Sammelpaket `ctrlutils` 0.5 stecken `cufileops` **0.6**, `cuundo` **0.5**, `cusync` **0.3** und `cufilter` **0.2**; `cumonitor` 0.2 ist unverändert. `cufileops` verlangt `tkutils::tkudialog 0.3` und `tkutils::tkufilelist 0.2`; ein älteres tkutils mit Dialog 0.2 reicht nicht. Was neu ist, steht in den Kapiteln ctrlutils und cufileops und Undo.

Zur Zählung: Im CHANGELOG von ctrlutils stehen `cufileops` 0.4 und `cuundo` 0.3 (Texte über msgcat, Eintrag) unter "## 0.3". Sie kamen, als das Sammelpaket schon 0.3 hieß, und bekamen keine eigene Nummer. Sie gehören zu ctrlutils 0.3, wie ausgeliefert. Ab 0.4 steigt mit jedem Modul auch die Nummer des Sammelpakets.

Die beiden anderen Bibliotheken bringen für den Stack wenig Neues:

- tkutils 0.49.0: `tkutablelist` 0.5 (Zeilen-IDs, Sortieren über `tablelist::sortByColumn`, CSV-Export nur sichtbarer Zeilen und Spalten in UTF-8), die Tablelist-Helfer `tkutlclip` 0.3 (Strg+C greift jetzt), `tkutlfind` 0.3 (Treffer bleiben nach dem Sortieren richtig), `tkutlsort` 0.3, `tkutlstate` 0.4, `tkutltree` 0.3, neu `tkutlfilter`, `tkutlgroup`, `tkutldrag` und `tkusysinfo`. Der Explorer von ctrlutils nutzt `tkutablelist` und `tkutlfind`; beide laufen ohne Änderung an ctrlutils, die Suche in der Liste profitiert vom neuen `tkutlfind`.
- tclutils 0.69.0: `tucsv` 0.2 (Excel-Profile, BOM, Kodierung), `tusqlite` 0.2 (Parameter-Dict, `update`, `delete`), neu `tusysinfo` und `tuedid`. ctrlutils benutzt keines davon.
- tkutils 0.48.0: `tkulauncher` 0.3 (mit `-scroll 1` bekommen auch nach `reload` neue Knöpfe die Mausrad-Bindung) und `tkuload` 0.3 (exportiert `widgetByName`, `setValue`, `getValue`, `fill`, `collect`). ctrlutils benutzt beide nicht.
- tclutils 0.68.0: Korrekturen in 76 Modulen, jedes mit neuer Nummer (Liste im CHANGELOG). Für den Stack zählen `tuprovider` 0.2 (`head` des lokalen Providers liefert den Inhalt), `tuprovider::sftp` 0.2 und `tuopen` 0.2 (unter Windows gehen URLs über `rundll32`, alle Befehle exportiert). tkutils und ctrlutils verlangen diese Module ohne Versionsnummer und laden 0.2 ohne Änderung. Die Fassung verlangt von ihnen nichts Neues.
- tclutils 0.67.0: `tupdftext` 0.1, der Text einer PDF-Datei. tkutils und ctrlutils benutzen es nicht.
- tclutils 0.66.0: `tutask` 0.2 (Korrekturen an `group`, `pool` und `-timeout`; verlangt `tclutils::common` 0.2). tkutils und ctrlutils benutzen `tutask` nicht.
- tkutils 0.47.0: `tkufiledialog` 0.2 (Doppelklick auf eine Datei lieferte `""`). Die `tkutl*` -Helfer laden jetzt `tablelist_tile` wie `tkutablelist`; vorher scheiterte `tkutils` zusammen mit `tkutltools` an `can't set "usingTile"` (laut CHANGELOG). Optionsfehler in zehn Modulen folgen `tkuopts` (`unknown option "…"` plus Zeile `Known:`).

2.2 Einmal Setup

Die Datei `tools/setup.tcl` ist in tclutils und tkutils **identisch**. Wer eine davon sourct, macht **beide** Bibliotheken auffindbar — ctrlutils aber nicht. Den Modulpfad für ctrlutils selbst ergänzen:

```
source /pfad/zu/tclutils/tools/setup.tcl     ;# oder .../tkutils/tools/setup.tcl
tcl::tm::path add /pfad/zu/ctrlutils/lib/tm

package require Tk
package require tclutils
package require tkutils
package require ctrlutils
```

Ohne die zweite Zeile endet `package require ctrlutils` mit `can't find package ctrlutils`. Einzelne Controller lassen sich auch allein laden (`package require ctrlutils::cufilter`). Das Sammelpaket `ctrlutils` braucht Tk und ein Display; ohne Tk nur die Tk-freien Controller einzeln laden (siehe Stack verstehen).

`ctrlutils` 0.3 lädt `tkutils::tkufilelist` nicht mit, obwohl `cufileops` dessen Prozeduren aufruft. Bis ctrlutils 0.3 nötig: die Liste selbst mit `package require tkutils::tkufilelist` laden — sonst endet der erste Aufruf mit `invalid command name "::tkutils::tkufilelist::widget"`.

Seit ctrlutils 0.4 lädt `cufileops` `tkutils::tkufilelist 0.2` selbst. Geprüft (Tcl 8.6.14 und 9.0.4, Linux, Xvfb): Nach `package require Tk` und `package require ctrlutils` ist `tkutils::tkufilelist` 0.2 geladen; mit ctrlutils 0.3 ist es das nicht.

Geprüft (Tcl/Tk 8.6.14 und 9.0.4, Linux, Xvfb): Setup wie oben lädt tclutils 0.69.0, tkutils 0.49.0 und ctrlutils 0.5 mit `cufileops` 0.6, `cuundo` 0.5, `cufilter` 0.2, `cumonitor` 0.2, `cusync` 0.3, dazu `tkufilelist` 0.2, `tkutablelist` 0.5, `tuprovider` 0.2 und `tuopen` 0.2.

`setup.tcl` sucht je Bibliothek in dieser Reihenfolge und nimmt nur Verzeichnisse, die existieren:

Rang	Ort
1	TCLUTILS_TM / TKUTILS_TM (Override, gewinnt)
2	<repo>/lib/tm des Pakets, aus dem setup.tcl stammt
3	<tcl-install>/<pkg>/lib/tm neben der Tcl-Bibliothek
4	/usr/local/share/tcltk/<pkg>/lib/tm
5	$XDG_DATA_HOME bzw. $HOME/.local/share/tcltk/<pkg>/lib/tm
6	Geschwisterordner <pkg>/lib/tm
7	Geschwisterordner <pkg>-*/lib/tm (höchste Version)

Fehlt ein Paket wirklich, meldet das erst `package require` — so gewollt.

Unter Windows liegen die beiden Treffer, die gemeint sind, in `TCLUTILS_TM` und in `lib/tm` neben der gesourcten Datei. Der Kandidat `/usr/local/share/tcltk/...` wird auf dem aktuellen Laufwerk gesucht. Der Benutzerpfad gilt nur, wenn `HOME` oder `XDG_DATA_HOME` gesetzt ist. `USERPROFILE` liest das Skript nicht.

2.3 In eine Tcl-Installation legen

Statt `setup.tcl` lassen sich alle drei Bäume in das site-tcl-Verzeichnis einer Tcl-Installation kopieren: `<tcl>/lib/tcl8/site-tcl` für Tcl 8.6, `<tcl>/lib/tcl9/site-tcl` für Tcl 9. Allgemein liegt es neben der Tcl-Bibliothek: `[file dirname [info library]]/tcl8/site-tcl`. Debian nimmt dafür mehrere Orte, darunter `/usr/share/tcltk/tcl8/site-tcl` und `/usr/lib/tcltk/tcl8/site-tcl` (laut `tcl::tm::path list` unter Tcl 8.6.14). Dieses Verzeichnis steht schon im Standard-Modulpfad, danach genügt `package require`.

Kopiert wird jeweils der **Inhalt** von `lib/tm` mit Unterordnern:

```
site-tcl/
    tclutils-0.69.0.tm
    tclutils/            tu*-Module, Unterordner tuprovider/
    tkutils-0.49.0.tm
    tkutils/
    ctrlutils-0.5.tm
    ctrlutils/           cufileops-0.6, cufilter-0.2, cumonitor-0.2,
                         cusync-0.3, cuundo-0.5
```

Geprüft (Tcl/Tk 8.6.14 und 9.0.4, Linux, Xvfb): genau diese Ablage in einem Testverzeichnis, über `TCL8_6_TM_PATH` bzw. `TCL9_0_TM_PATH` eingehängt, lädt alle drei Sammelpakete. Ein echtes `lib/tcl9/site-tcl` ist nicht geprüft.

tical gehört nicht in diese Ablage: Seine Pakete stehen in einem `pkgIndex.tcl` und werden

über `auto_path` gefunden. Der ganze Ordner `tical/` gehört deshalb in ein Verzeichnis, das im `auto_path` steht (bei Tcl 9 unter `/opt/tcl9` etwa `/opt/tcl9/lib`, unter Debian etwa `/usr/local/share/tcltk`), oder sein Elternverzeichnis kommt in `TCLLIBPATH`. Geprüft (Tcl 8.6.14 und 9.0.4, Linux): mit dem Elternverzeichnis in `TCLLIBPATH` lädt `package require tical` die Version 1.1. Mehr im Kapitel Kalender: tical.

Keines der drei Repos bringt ein Installationsskript mit. Wer eins schreibt: Zielverzeichnis aus der Tcl-Version ableiten, die drei `lib/tm`-Bäume rekursiv kopieren, alte Dateien derselben Module vorher entfernen. Das ist nötig, weil Modulnamen die Version tragen: Beim bloßen Drüberkopieren bleibt die alte Datei neben der neuen liegen. Beim Wechsel auf dieses Pairing sind das:

Lib	**alte Dateien löschen**
ctrlutils 0.5	ctrlutils-0.4.tm; in ctrlutils/: cufileops-0.5.tm, cuundo-0.4.tm, cusync-0.2.tm, cufilter-0.1.tm, dazu cufileops-0.4.tm, cuundo-0.3.tm, cusync-0.1.tm (sie lagen noch im Baum von 0.4); von 0.3 aus statt ctrlutils-0.4.tm die Datei ctrlutils-0.3.tm, von noch älteren Ständen auch cufileops-0.3.tm, cuundo-0.2.tm
tkutils 0.49.0	tkutils-0.48.0.tm; in tkutils/: tkutablelist-0.4.tm, tkutlclip-0.2.tm, tkutlfind-0.2.tm, tkutlsort-0.2.tm, tkutlstate-0.3.tm, tkutltools-0.2.tm, tkutltree-0.2.tm, tkuical-0.2.tm, tkuvcard-0.2.tm, tkuldif-0.2.tm, tkuini-0.2.tm, tkutical-0.3.tm, tkunotes-0.2.tm, tkueditor-0.2.tm, tkubase64-0.1.tm, tkutlfooter-0.3.tm, tkulauncher-0.3.tm, tkuhexedit-0.1.tm (von 0.47.0 aus statt tkutils-0.48.0.tm die Datei tkutils-0.47.0.tm, dazu tkulauncher-0.2.tm, tkuload-0.2.tm; von 0.46.0 aus statt tkutils-0.47.0.tm die Datei tkutils-0.46.0.tm, dazu tkufiledialog-0.1.tm, tkutlclip-0.1.tm, tkutlfind-0.1.tm, tkutlsort-0.1.tm, tkutltools-0.1.tm, tkutltree-0.1.tm, tkutlfmt-0.2.tm, tkutlfooter-0.2.tm, tkutlstate-0.2.tm)

Lib	alte Dateien löschen
tclutils 0.69.0	tclutils-0.68.0.tm, in tclutils/tucsv-0.1.tm, tusqlite-0.1.tm, tuholiday-0.1.tm, tuzip-0.1.tm, tudiff-0.1.tm, tubookmark-0.2.tm, tuical-0.1.tm, tuvcard-0.1.tm, tuini-0.1.tm, tupatch-0.2.tm, tusplit-0.2.tm, tutee-0.2.tm, tumd-0.1.tm, tuiconv-0.1.tm, tucat-0.1.tm, tuldif-0.2.tm, tucsplit-0.1.tm, tuhead-0.1.tm, tutail-0.1.tm, tugrep-0.1.3.tm, tustr-0.2.tm, tuer-0.3.tm, tugantt-0.2.tm, tuxychart-0.1.tm, tusankey-0.1.tm, tudate-0.2.tm, tunumany-0.2.tm, tubase64-0.1.tm, tucrc-0.1.tm, tudav-0.2.tm, tuflow-0.3.tm, tuc4-0.1.tm, tugit-0.2.tm, turadar-0.2.tm, tusort-0.1.tm, tuuniq-0.1.tm, tucut-0.1.tm, tupaste-0.2.tm, tujoin-0.1.2.tm, tushuf-0.2.tm, tucolumn-0.1.tm, tufold-0.2.tm, tufmt-0.1.tm, tutestrun-0.1.tm, tuarchitecture-0.2.tm, tustate-0.1.tm, tublock-0.1.tm, tunotes-0.1.tm, tumindmap-0.1.tm, common-0.2.tm, tused-0.1.3.tm, tusettings-0.1.tm, tuimage-0.1.tm, tuprovider-0.2.tm, tutreemap-0.2.tm, tuclass-0.2.tm, tupacket-0.2.tm, tujourney-0.2.tm, tukanban-0.1.tm, tutimeline-0.2.tm, tupie-0.2.tm, tuquadrant-0.1.tm, tusequence-0.2.tm, tudiagram-0.5.tm, tusvg-0.3.tm, tupngdraw-0.13.tm, tupng-0.5.tm, tuappinfo-0.2.tm, tuodf-0.1.tm; von 0.67.0 aus statt tclutils-0.68.0.tm die Datei tclutils-0.67.0.tm, dazu die alten Dateien der 76 Module mit neuer Nummer in 0.68.0 (Liste im CHANGELOG), darunter tuprovider-0.1.tm, tuopen-0.1.tm, tuprovider/sftp-0.1.tm; von älteren Ständen auch tclutils-0.66.0.tm, tclutils-0.65.0.tm und tutask-0.1.tm

Danach gilt: Eine installierte Kopie kann einen Arbeitsbaum verdecken — siehe den Test-Abschnitt unten.

2.4 Tests

Jede Bibliothek bringt `tests/all.tcl` mit. Für ctrlutils:

```
cd ctrlutils
tclsh tests/all.tcl
tclsh tests/all.tcl -match 'cufileops-*'
```

Optionen wie `-match` gehen an jede Testdatei. Mit `-match 'cufileops-*'` laufen alle acht Dateien, aber nur in `cufileops.test` passt ein Test; die übrigen melden alles als übersprungen.

Der Runner ist tutestrun aus tclutils (seit 0.64). `all.tcl` holt ihn aus dem eigenen `lib/tm`, aus `$TCLUTILS_TM` oder aus dem Geschwister-Checkout `tclutils*/lib/tm` — nie aus einer installierten Kopie. Findet es ihn nicht: `all.tcl: tclutils::tutestrun not found (set TCLUTILS_TM)`, Exit 2 (geprüft mit einer Kopie von `tests` und `lib` ohne Geschwister-Checkouts).

Jede `.test`-Datei läuft in einem eigenen Prozess. Vor dem Lauf druckt der Runner ein Banner, aus welcher Datei ctrlutils, tkutils und tclutils geladen werden, zum Beispiel:

```
Modules resolve to:
  ctrlutils  0.5      .../ctrlutils/lib/tm/ctrlutils-0.5.tm
  tkutils    0.49.0   .../tkutils/lib/tm/tkutils-0.49.0.tm
  tclutils   0.69.0   .../tclutils/lib/tm/tclutils-0.69.0.tm
```

Liegt eine zweite Version daneben, steht dahinter `(also: <version>)`, etwa `(also: 0.4)`, solange `ctrlutils-0.4.tm` neben `ctrlutils-0.5.tm` liegt. Alte Dateien der einzelnen Module sieht der Runner nicht; ein Baum von ctrlutils 0.4 enthielt noch `cufileops-0.4.tm`, `cuundo-0.3.tm` und `cusync-0.1.tm`, ohne dass der Banner es zeigte. Gesetzte Variablen wie `TCL8_6_TM_PATH` oder `TCLUTILS_TM` meldet je eine Zeile `note: <VAR> is set: …`.

Eigene Bäume stehen vorn: der Runner nimmt sie aus `TCL8_6_TM_PATH` / `TCL9_0_TM_PATH` heraus und setzt sie im Testprozess an die Spitze. Eine direkt gestartete Testdatei ohne Runner hat diesen Schutz nicht.

Zeitgrenze je Datei: CTRLUTILS_TESTGRENZE in Sekunden, Standard

1. tclutils und tkutils folgen demselben Schema mit

`TCLUTILS_TESTGRENZE` und `TKUTILS_TESTGRENZE`. Am Ende steht `All test files passed` oder eine Liste `Files with problems:` mit dem Grund je Datei: `N failed`, `no summary`, `exit code / error`, `hang after Ns`, `could not start`. Dann ist der Exit-Code 1.

Geprüft mit ctrlutils 0.5, tkutils 0.49.0 und tclutils 0.69.0 (Linux, Xvfb), unter Tcl/Tk 8.6.14 und 9.0.4 gleich:

- 8 Testdateien, **91** Tests, alle grün (`cufileops` 27, `cufilter` 7, `cumonitor` 6, `cusync` 14, `cuundo` 15, `headers` 1, `smoke` 20, `versions` 1).
- Neu gegenüber ctrlutils 0.4 (82 Tests): `cusync-4.1` bis `4.3` (Auflisten scheitert), `cufileops-13.1` (Cut nach Teilfehler), `cufileops-13.2` (`refresh`), `cuundo-12.1`, `12.2` (`-refresh`), `cufilter-10.1`, `10.2` (Fehler im Filter).
- Neu in ctrlutils 0.4 gegenüber 0.3 (76 Tests): `cuundo-1.10` (Redo), `cusync-3.6` (Optionen), `cusync-3.7` (mtime), `cuundo-11.1` (Dialog-Parent, msgcat), `cufileops-12.1` (owSetApp), `cufilter-9.1` (msgcat).
- `CTRLUTILS_TESTGRENZE=1` meldet `cumonitor.test (hang after 1s)` (geprüft mit ctrlutils 0.3).
- `cumonitor.test` arbeitet im System-Temp; ein abgebrochener Lauf lässt kein `cumon/` mehr im Baum.

Ohne Display werden die Tests von `cufileops` und `cufilter` übersprungen. In ctrlutils 0.3 scheitert dann `smoke.test` in `explorer-0-load` (`couldn't connect to display`), Exit 1. Seit 0.4 überspringt `smoke.test` diesen Test: 20 übersprungen, 0 Fehler. Für die Tk-Tests also ein Display nehmen (unter Linux ohne Bildschirm: `xvfb-run -a tclsh tests/all.tcl`).

Die Suiten der beiden anderen Bibliotheken (Linux, Xvfb), jeweils 0 Fehler:

Suite	Tests	Tcl 8.6.14	Tcl 9.0.4
tclutils 0.69.0	2639	2614 bestanden, 25 übersprungen	2622 bestanden, 17 übersprungen
tkutils 0.49.0	947	935 bestanden, 12 übersprungen	944 bestanden, 3 übersprungen

Die tkutils-Suite lief mit tical als Geschwister-Checkout; ohne tical überspringen

`tkutical.test` und `tkumonthcanvas.test` ihre Tests. Die Suite von tical selbst steht im Kapitel Kalender: tical.

Unter macOS (Darwin 27, arm64, Tcl/Tk 9.0.4 mit Aqua) liefen die drei Suiten vor den Schreibfehler-Tests ohne Fehler:

Suite	Tests	bestanden	übersprungen
tclutils 0.69.0	2582	2567	15
tkutils 0.49.0	938	896	42
ctrlutils 0.4	82	82	0

Unter Windows liefen die drei Suiten im selben Stand ebenfalls ohne Fehler, Tcl 8.6.17 mit der Systemkodierung cp1252 und Tcl 9.0.4:

Suite	Tests	Tcl 8.6.17	Tcl 9.0.4
tclutils 0.69.0	2582	2545 bestanden, 37 übersprungen	2549 bestanden, 33 übersprungen
tkutils 0.49.0	938	927 bestanden, 11 übersprungen	935 bestanden, 3 übersprungen
ctrlutils 0.4	82	82 bestanden	82 bestanden

Die Übersprünge unter Windows kommen von der Plattform (`unix`) und von fehlenden Paketen (Zipfs, SFTP, HTTPS).

2.5 Zusammenfassung

- Pairing 0.69.0 / 0.49.0 / 0.5; im Sammelpaket ctrlutils 0.5 stecken `cufileops` 0.6, `cuundo` 0.5, `cusync` 0.3 und `cufilter` 0.2. tical 1.1.1 optional daneben.
- `setup.tcl` kennt tclutils und tkutils; ctrlutils per `tcl::tm::path add …/ctrlutils/lib/tm`; `tkufilelist` bis ctrlutils 0.3 selbst laden, ab 0.4 lädt `cufileops` es.
- Alternativ alle drei `lib/tm`-Bäume nach `site-tcl` kopieren, alte Moduldateien vorher entfernen.
- `tclsh tests/all.tcl`; Banner lesen, Grenze über `*_TESTGRENZE`.

Nächstes Kapitel: ctrlutils

3 ctrlutils

3.1 Fünf Controller

Modul	Rolle	Tk
cufileops 0.5	Kontextmenü und Tasten an der Dateiliste: neu, öffnen, öffnen mit, umbenennen, kopieren, ausschneiden, einfügen, löschen	ja

Modul	Rolle	Tk
cuundo 0.4	Undo/Redo für die Operationen von cufileops	lädt ohne Tk; Tasten nur mit -list
cufilter 0.1	eigene Filterzeile über der Liste	ja
cumonitor 0.2	Verzeichnis per Polling überwachen, Callback bei Änderung	nein (nur after)
cusync 0.2	zwei Bäume vergleichen und einseitig spiegeln	nein

```
package require ctrlutils
# oder einzeln: package require ctrlutils::cufileops 0.6
```

`package require ctrlutils` lädt alle fünf und braucht dafür Tk. Seit ctrlutils 0.4 lädt `cufileops` auch `tkutils::tkufilelist` 0.2; bis 0.3 musste die App das selbst tun. Docs: `ctrlutils/docs/<modul>.md`, Manpages: `ctrlutils/man/mann/<modul>.n`. Suite: **91** Tests in 8 Dateien (geprüft, Tcl/Tk 8.6.14 und 9.0.4, Linux, Xvfb).

Neu in ctrlutils 0.5, kurz:

- `cusync`: Ein Verzeichnis, das sich nicht auflisten lässt, ist ein Fehler `{CTRLUTILS CUSYNC LIST side dir}`. Bis 0.4 galt die Seite als leer, und der Spiegel kopierte jede Datei der Quelle darüber.
- `cufileops`: Einträge eines Cut, die sich nicht kopieren ließen, bleiben in der Zwischenablage. Neu `refresh $f`.
- `cuundo`: Option `-refresh`; ein scheiterndes Auffrischen ist ein Fehler statt still verschluckt.
- `cufilter`: Ein Filter, der sich nicht setzen lässt, markiert die Zeile und bleibt über `lastError` lesbar.

Neu in ctrlutils 0.4, kurz:

- `cuundo`: Ein gescheitertes Redo bleibt auf dem Redo-Stapel. Die Fehlerdialoge bekommen `-parent`, die Gründe laufen über msgcat. `tclutils::tuprovider` lädt es nicht mehr selbst.
- `cufileops`: lädt `tkufilelist` 0.2; neu `owSetApp` für den App-Namen des "Open with"-Registers.
- `cufilter`: "Filter:" und "x" laufen über msgcat.
- `cusync`: prüft die Optionen von `apply`; die mtime überträgt es nur zwischen zwei lokalen Providern.
- Explorer: Die Startmeldung bei fehlendem Modul nennt auch den Modulpfad.

Jeder Controller hängt nur an der Provider-Schnittstelle von tclutils und an tkutils-Widgets — nie an einer bestimmten App. Zustand liegt je Sicht unter einem Schlüssel `$f` (z. B. dem Tab-Frame); mehrere Sichten bleiben so getrennt. Undo und Liste finden sich über dasselbe `$f`: cufileops und cuundo für eine Sicht mit demselben Schlüssel installieren.

3.2 Abhängigkeit zur View

`cufileops` 0.5 braucht `tkucontextmenu`, `tkudialog` 0.3 und `tkufilelist` 0.2; `tuopen` (Öffnen mit der Standard-App) und `msgcat` sind optional. Es spielt mit `tkutablelist` ab 0.4

(Standard `-exportselection 0`), damit die Mehrfachauswahl beim Rechtsklick nicht verloren geht. `cufileops` 0.4 aus ctrlutils 0.3 rief `tkufilelist` auf, ohne es zu laden. Details: cufileops und Undo.

3.3 Weiter

Die drei kleineren Module haben eigene Kapitel: Filter, Verzeichnis beobachten, Spiegeln. Dateioperationen und Undo stehen im Kapitel cufileops und Undo.

3.4 Zusammenfassung

- Fünf Module. cufileops, cuundo und cumonitor hängen an vorhandenen Pfaden. cufilter baut eine eigene Zeile. cusync rechnet ohne Tk.
- Das Sammelpaket braucht Tk. cusync, cumonitor und cuundo laden auch ohne.
- ctrlutils 0.4 lädt `tkufilelist`, behält ein gescheitertes Redo, prüft die Optionen von `cusync` und nimmt einen eigenen App-Namen für "Open with".
- ctrlutils 0.5 macht einen Fehler beim Auflisten in `cusync` laut und behält nach einem Teilfehler den Rest eines Cut.

Nächstes Kapitel: Filter

4 Filter: cufilter

cufilter 0.2 legt selbst Widgets an. Die übrigen Controller hängen Verhalten an vorhandene Pfade. `widget $path $listwidget` baut einen `ttk::frame` mit Label "Filter:", Eingabefeld und Leeren-Knopf (Aufschrift "x") und gibt `$path` zurück. Packen muss die App.

```
::tkutils::tkufilelist::widget .list -dir [pwd]
::ctrlutils::cufilter::widget .filter .list
pack .filter -side top -fill x
pack .list   -side top -fill both -expand 1
```

- Jeder Tastendruck filtert sofort.
- Reiner Text wird zu `*text*` (Teilstring), Text mit `*` oder? gilt als Glob. Groß-/Kleinschreibung egal: `tkufilelist` vergleicht mit `string match -nocase` ("A" findet `a.txt`).
- Verzeichnisse bleiben immer sichtbar, Navigation geht weiter.
- Escape oder der Knopf leeren die Zeile (`cufilter::clear $path`), der Fokus geht zurück an die Liste.
- Seit ctrlutils 0.4 gehen "Filter:" und "x" über msgcat (Katalog im Namensraum `::ctrlutils`, wie bei cufileops). Geprüft (Tcl/Tk 8.6.14, Linux, Xvfb): mit `mcset {} Filter: Suche:` und `mcset {} x X` zeigt die Zeile `Suche:` und `X`. In ctrlutils 0.3 bleiben beide englisch.
- Lässt sich der Filter nicht setzen (die Liste ist weg oder keine `tkufilelist`), geht das Eingabefeld in den ttk-Zustand `invalid`, `$path` bekommt `<<CufilterError>>`, und `cufilter::lastError $path` liefert die Meldung. Ein Dialog erscheint nicht, weil der Filter bei jedem Tastendruck läuft. Bis 0.1 verschwand der Fehler still.

Voraussetzung: `tkufilelist` mit Option `-filter` / Methode `setFilter`. Die Liste selbst steht im Handbuch tkutils.

4.1 Merkliste

- `widget` liefert den Pfad und packt ihn nicht.
- Text ohne `*` und? sucht als Teilstring.
- Verzeichnisse bleiben sichtbar.
- Die beiden Aufschriften kommen seit 0.4 aus msgcat.
- Ein Fehler beim Filtern steht in `lastError` (seit 0.2).

Nächstes Kapitel: Verzeichnis beobachten

5 Verzeichnis beobachten: cumonitor

cumonitor 0.2 pollt `list` des Providers und vergleicht eine Signatur aus Name, Typ, Größe und mtime. Kein inotify. Polling ist der Weg, der lokal, im ZIP und auf entfernten Providern gleich funktioniert. Das Modul braucht kein Tk, aber eine Ereignisschleife.

```
::ctrlutils::cumonitor::install $f $prov -interval 2000 \
    -onchange [list ::app::onDirChanged $f]
::ctrlutils::cumonitor::watch $f $dir    ;# neue Basis, feuert nicht
# nach eigenen Änderungen der App:
::ctrlutils::cumonitor::poke $f
::ctrlutils::cumonitor::stop $f          ;# beim Schließen des Tabs
```

- `-interval` in Millisekunden, Standard 2000.
- `-onchange` bekommt das geänderte Verzeichnis angehängt. Ein Fehler im Callback wird verschluckt; das Polling läuft weiter.
- Eine unbekannte Option wirft `{CTRLUTILS CUMONITOR OPTION -name}` und nennt die bekannten. Vor 0.2 war das ein Fehler ohne Errorcode.
- Die erste Momentaufnahme ist die Basis; erst eine echte Abweichung feuert. `watch` setzt die Basis still neu, Navigieren sieht also nicht wie eine Änderung aus.
- `watch` vor `install` ist ein Fehler: `cumonitor: <f> not installed`. Ist im Prozess noch gar kein Monitor installiert, melden `watch`, `poke` und `stop` stattdessen `can't read "state": no such variable` (geprüft, Tcl 8.6.14 und 9.0.4). `stop` ist also erst nach dem ersten `install` gefahrlos.
- Ein Verzeichnis, das nicht mehr lesbar ist, zählt als Änderung.
- Ohne `poke` nach eigenen Operationen meldet der Monitor sie zurück.
- `stop` ist wiederholbar und verwirft den Zustand.

Geprüft (Tcl 8.6.14 und 9.0.4, Linux): unter `tclsh` mit `vwait` und `-interval 200` feuert `-onchange` nach dem Anlegen einer Datei und bekommt das beobachtete Verzeichnis angehängt.

Der Explorer setzt das Intervall auf 2000 ms und ruft beim Schließen eines Tabs `stop`. Das steht im Kapitel Explorer und Dateiblick.

5.1 Merkliste

- Signatur: Name, Typ, Größe, mtime.
- `watch` feuert nicht. `poke` schluckt die eigenen Änderungen.
- Ein Fehler im Callback stoppt das Polling nicht.
- Ohne Tk laden, mit Ereignisschleife laufen lassen.

Nächstes Kapitel: Spiegeln: cusync

6 Spiegeln: cusync

cusync 0.2 ist Tk-frei und läuft unter `tclsh`. Jede Seite hat ihren eigenen Provider. Lokal, ZIP, WebDAV und SFTP lassen sich mischen. Der Ablauf ist compare, dann plan, dann apply. Einseitig und ohne Löschen: auf dem Ziel wird nichts entfernt. Eine Option zum Löschen gibt es nicht. Den Dialog liefert der Explorer, nicht cusync.

```
package require tclutils::tuprovider
package require ctrlutils::cusync

set pL [::tclutils::tuprovider open local]
set pR [::tclutils::tuprovider open local]
set diffs [::ctrlutils::cusync::compare $pL $dirL $pR $dirR]
set plan  [::ctrlutils::cusync::plan $diffs LR]
set res   [::ctrlutils::cusync::apply $pL $dirL $pR $dirR $plan]
```

Geprüft (Tcl 8.6.14 und 9.0.4, Linux) mit links `a.txt` und `sub/b.txt`, rechts nur `nur-rechts.txt`: vier Diffs (`a.txt`, `sub`, `sub/b.txt` als `onlyLeft`, `nur-rechts.txt` als `onlyRight`), drei Aktionen im Plan, Ergebnis `done 3 errors {}`. Ein zweites `compare` danach meldet `a.txt same`, `nur-rechts.txt onlyRight` und `sub/b.txt same`; das Verzeichnis `sub`, jetzt auf beiden Seiten, steht nicht mehr in der Liste. Die rechte Datei bleibt stehen.

6.1 compare

`compare` liefert je Eintrag ein Dict. Eltern stehen vor den Kindern.

Schlüssel	Inhalt
rel	Pfad relativ zu den beiden Wurzeln, /, ohne führenden Schrägstrich
name	Blattname
status	siehe Tabelle
ltype	file, dir oder "", wenn links fehlend
rtype	file, dir oder "", wenn rechts fehlend

Status	Bedeutung
onlyLeft	nur links
onlyRight	nur rechts
newerLeft	beide, links neuer
newerRight	beide, rechts neuer
diff	beide, gleiche oder fehlende mtime, andere Größe
same	beide, gleiche mtime und Größe

Status	Bedeutung
conflict	Datei gegen Verzeichnis

Verzeichnisse, die nur auf einer Seite liegen, werden betreten. Jede Datei darunter bekommt einen eigenen Eintrag. Ein Verzeichnis, das auf beiden Seiten liegt, hat selbst keinen Eintrag; nur sein Inhalt erscheint.

`same` vergleicht nur Größe und mtime, nicht den Inhalt. Zwei Dateien mit gleichen Angaben und anderem Inhalt gelten als gleich.

Lässt sich ein Verzeichnis, das auf einer Seite liegt, nicht auflisten, endet `compare` mit dem Fehler `{CTRLUTILS CUSYNC LIST side dir}` (`side` ist `L` oder `R`). Bis ctrlutils 0.4 (`cusync` 0.2) galt diese Seite als leer: Scheiterte das Auflisten des Ziels, wurde jede Datei der Quelle `onlyLeft`, und `apply` kopierte sie über das Ziel, auch über neuere Dateien. Der Explorer zeigt den Fehler in der Statuszeile des Dialogs.

6.2 plan

`plan` macht aus den Diffs Kopieraktionen `{op copy rel … type file|dir}` für eine Richtung, `LR` oder `RL`. Kopiert wird, was im Ziel fehlt oder dort älter oder abweichend ist. Die Seite, die nur im Ziel existiert, kommt nicht herüber. `conflict` auch nicht. Ein `LR`-Plan übergeht `onlyRight`, ein `RL`-Plan übergeht `onlyLeft`.

6.3 apply und applyOne

`apply` führt den Plan aus und liefert `done N errors {…}`. Ein Fehlertext beginnt mit dem relativen Pfad, zum Beispiel `fehlt.txt (read): couldn't open …` oder `… (write): …`. Ein Fehler bricht den Rest nicht ab.

`-ondir` und `-onfile` bekommen je Aktion den relativen Pfad angehängt, vor der Aktion. Ein Fehler im Callback wird übergangen. In `cusync` 0.1 aus ctrlutils 0.3 prüft `apply` andere Optionen nicht. Ein Tippfehler bleibt still. Seit 0.2 ist eine unbekannte Option ein Fehler `unknown option "-x"` mit der Zeile `Known: -onfile -ondir` und dem Errorcode `{CTRLUTILS CUSYNC OPTION -x}`. Eine Option ohne Wert meldet `missing value for option "-onfile"` mit dem Errorcode `{CTRLUTILS CUSYNC OPTION}` (geprüft, Tcl 8.6.14 und 9.0.4, Linux, Xvfb).

`applyOne` führt eine einzelne Aktion aus und liefert `ok 1` oder `ok 0 error …`. Eine Oberfläche kann den Plan per `after` schrittweise abarbeiten. Vor einer Datei legt `applyOne` per `mkdir` das Elternverzeichnis am Ziel an. Ein Fehler dabei bleibt still.

Die mtime der Quelle überträgt `applyOne` mit `file mtime`. Das wirkt nur bei lokalen Pfaden, sonst bleibt es still. In ctrlutils 0.3 kann dabei bei einem ZIP-, DAV- oder SFTP-Ziel eine gleichnamige lokale Datei die mtime bekommen. Seit ctrlutils 0.4 überträgt `applyOne` die mtime nur, wenn beide Provider der lokale sind (Test `cusync-3.7`).

6.4 Merkliste

- compare, plan, apply. Richtung `LR` oder `RL`.
- Nichts wird gelöscht.

- Ein Fehler beim Auflisten bricht `compare` ab (seit `cusync` 0.3).
- `apply` sammelt Fehler und läuft weiter. `applyOne` nimmt eine Aktion.
- mtime nur zwischen zwei lokalen Providern.

Nächstes Kapitel: cufileops und Undo

7 cufileops und Undo

7.1 install

```
ctrlutils::cufileops::install $f $prov $listwidget $refreshcmd ?-onop script?
```

- `$f` — Identität der Sicht (z. B. Tab-Frame); aller Zustand hängt daran
- `$prov` — Provider
- `$listwidget` — ein `tkufilelist`
- `$refreshcmd` — Skript nach jeder erfolgreichen Änderung (Liste neu lesen)
- `-onop script` — optional; bekommt nach jeder erfolgreichen Operation einen Operationssatz (Dict) angehängt. Einziger Abnehmer bisher: cuundo.

Eine unbekannte Option ist ein Fehler mit Errorcode `{CTRLUTILS CUFILEOPS OPTION <opt>}` und nennt die bekannten. Eine Option ohne Wert meldet `missing value for option "-onop"` (geprüft, Tcl 8.6.14 und 9.0.4). Bis 0.1 verschluckte `install` einen Tippfehler wie `-onpo` still — Undo war dann einfach aus.

Das Kapitel gilt für `cufileops` 0.5 und `cuundo` 0.4 aus ctrlutils 0.4. Die Stufen davor:

ctrlutils	cufileops / cuundo	neu
0.3	0.3 / 0.2	Mehrfachauswahl beim Rechtsklick, Dialog 0.3
0.3 (Nachtrag)	0.4 / 0.3	Texte über msgcat
0.4	0.5 / 0.4	tkufilelist geladen, owSetApp, Redo bleibt erhalten, cuundo-Dialoge mit -parent, cuundo-Gründe über msgcat

Das Verhalten der Operationen selbst ist über alle Stufen gleich.

7.2 Menü und Tasten

cufileops baut das Kontextmenü bei jedem Rechtsklick neu, aus den caps des Providers. Ein Eintrag erscheint nur, wenn die Operation gelingen kann:

Eintrag	erscheint, wenn
New Folder	mkdir
Open	Datei, get
Open with <App>	Datei, get, Standard-App für die Endung registriert
Open with...	Datei, get

Eintrag	erscheint, wenn
Rename	move
Copy	Datei: get; Verzeichnis: list und get
Cut	wie Copy, zusätzlich delete an der Quelle
Delete	delete
Paste	Zwischenablage voll; put, bei Verzeichnissen auch mkdir

Trifft nichts davon zu, zeigt das Menü nur den ausgegrauten Eintrag "(read-only)". Ein ZIP ist nicht automatisch "(read-only)": seine caps sind `get list stat`, auch mit `tuprovider` 0.2 aus tclutils 0.69.0 (geprüft, Tcl 8.6.14 und 9.0.4). Geprüft (Tcl/Tk 8.6.14, Linux, Xvfb):

- ZIP, Datei markiert: `Open`, `Open with...`, Trennlinie, `Copy`.
- ZIP, nichts markiert, Zwischenablage leer: nur "(read-only)".
- Lokal, Verzeichnis markiert: `New Folder`, `Rename`, `Copy`, `Cut`, `Delete`.
- Lokal, Zwischenablage voll, nichts markiert: `New Folder`, `Paste`.

Tasten auf dem Listenkörper: Entf (löschen), F2 (umbenennen), Strg+C, Strg+X, Strg+V. Strg+X und Entf auf einer Quelle ohne `delete` melden das per Dialog statt still nichts zu tun: `Cut is not available here (the source cannot delete).` bzw. `Delete is not available here (the source cannot delete).` Die übrigen Tasten tun ohne passende caps nichts.

Alle Dialoge von cufileops bekommen als `-parent` das Toplevel der Liste und erscheinen über dem eigenen Fenster, nicht mitten auf einem anderen Monitor. Seit `cuundo` 0.4 gilt das auch für dessen Fehlermeldungen (Toplevel der Liste aus `-list`; ohne `-list` kein `-parent`). In `cuundo` 0.3 aus ctrlutils 0.3 kamen sie ohne `-parent`.

7.3 Öffnen

- **Open** startet die Standard-App des Systems über `tclutils::tuopen` (Rückfall ohne tuopen: `xdg-open` / `open` / `cmd /c start`).
- **Open with...** fragt nach einem Programm. Die Auswahl merkt sich ein Register je Dateiendung; vorgeschlagen werden passende installierte Programme (Prüfung mit `auto_execok`).
- **Open with <App>** erscheint, sobald für die Endung eine Standard-App gesetzt ist — ein Klick, kein Dialog.

Das Register ist ein Tcl-Dict in der Datei `openwith.dict`, abgelegt über `tuopen::configFile` im Konfigurationsverzeichnis der App `explorer`, solange die App keinen eigenen Namen setzt. Eigene Apps setzen ihn einmal beim Start, vor dem ersten Menü:

```
::ctrlutils::cufileops::owSetApp meineapp
```

owSetApp (seit `cufileops` 0.5, ctrlutils 0.4) nimmt nur Buchstaben, Ziffern, `.`, `_` und `-`. Sonst ist es ein Fehler `invalid application name "…": use letters, digits, . _ -` mit dem Errorcode `{CTRLUTILS CUFILEOPS APPNAME <name>}`. Es verwirft das zwischengespeicherte Register; der nächste Zugriff liest die Datei der neuen App. Geprüft (Tcl/Tk 8.6.14 und 9.0.4, Linux, Xvfb): `owSetApp "mein app"` meldet den Fehler mit `CTRLUTILS CUFILEOPS APPNAME {mein app}`; nach `owSetApp meineapp` und `owAddApp .txt vi`

liegt das Register unter `$XDG_CONFIG_HOME/meineapp/openwith.dict`.

Bis ctrlutils 0.3 gab es nur die Variable `::ctrlutils::cufileops::owApp`. Jede App ohne eigenen Wert teilte sich das Register von `explorer`. Wer die Variable erst nach dem ersten Zugriff setzte, las weiter das alte Register. Neue Apps nehmen `owSetApp`.

Geprüft unter Linux: `$XDG_CONFIG_HOME/<app>/openwith.dict`, ohne `XDG_CONFIG_HOME` liegt es unter `$HOME/.config/<app>/`. Laut tuopen unter Windows `%APPDATA%\<app>`, unter macOS `$HOME/Library/Application Support/<app>` (nicht geprüft). Fehlt tuopen, landet es als `<app>-openwith.dict` in `$TMPDIR` bzw. `/tmp`.

Nur der lokale Provider öffnet eine vorhandene Datei direkt. Aus ZIP oder einem entfernten Provider holt cufileops die Bytes per `get` in ein prozesseigenes Temp-Verzeichnis `cufileops-<pid>-<µs>` unter `TMPDIR`, `TEMP`, `TMP` oder `/tmp`. Es verschwindet beim `exit` des Prozesses; Reste toter Prozesse oder älter als ein Tag räumt der nächste Aufruf ab. Geprüft (Tcl/Tk 8.6.14, Linux, Xvfb): `a.txt` aus einem ZIP lag beim Öffnen unter `$TMPDIR/cufileops-…/a.txt`, nach `exit` war das Verzeichnis weg. Doppelklick bleibt Sache der App.

7.4 Kopieren, Ausschneiden, Einfügen

Copy/Paste läuft über `get+put` im Controller, nicht über das `copy` des Providers — daher **cross-provider** (z. B. aus ZIP nach lokal), Verzeichnisse rekursiv. Die Zwischenablage gilt modulweit für alle Tabs und merkt sich den Quell-Provider. Bei Namensgleichheit wählt Paste `name (copy).ext`, dann `name (copy 2).ext` usw. und überschreibt nie. Scheitert ein Verzeichnis halb, löscht Paste das bereits Angelegte wieder.

Cut markiert zum **Verschieben**: Paste schreibt am Ziel und löscht dann die Quelle. Deshalb nur, wenn die Quelle `delete` kann — aus einem ZIP geht Copy, nicht Cut. Ein Cut wird von seinem Paste verbraucht; Einträge, die sich nicht kopieren ließen, bleiben seit `cufileops` 0.6 als Cut in der Zwischenablage, sodass Paste sie noch einmal versucht. Bis 0.5 war der ganze Cut nach einem Teilfehler weg. Ein Cut in dasselbe Verzeichnis desselben Providers überspringt Paste. Lässt sich die Quelle nach dem Kopieren nicht löschen, meldet Paste `Copied, but could not remove the source:` und behält die Kopie.

Geprüft (Tcl/Tk 8.6.14, Linux, Xvfb): `sub/` und `a.txt` aus einem ZIP in ein lokales Verzeichnis kopiert, samt `sub/c.txt`; zweimal `a.txt` eingefügt ergibt `a (copy).txt` und `a (copy 2).txt`; Cut von `b.txt` aus einem lokalen Tab in einen anderen verschiebt die Datei und leert die Zwischenablage.

7.5 Mehrfachauswahl beim Rechtsklick

Seit 0.3 behält ein Rechtsklick auf eine **bereits selektierte** Zeile die ganze Mehrfachauswahl (Dateimanager-Verhalten). Copy, Cut und Delete wirken dann auf alle markierten Einträge. Ein Rechtsklick auf eine nicht selektierte Zeile wählt nur diese.

Technik: Der Rechtsklick lässt tablelist die Zeile selbst wählen (synthetischer Button-1) und stellt danach die vorherige Auswahl wieder her, wenn die geklickte Zeile dazugehörte. Dafür braucht es `tkudialog` 0.3 und sinnvoll `tkutablelist` ab 0.4 mit `-exportselection 0` — sonst nimmt das Menü die PRIMARY-Auswahl und die Markierung verschwindet.

Geprüft (Tcl/Tk 8.6.14, Linux, Xvfb): Auswahl `0 2 3`, Rechtsklick auf Zeile 2 — Auswahl bleibt `0`

2 3; Rechtsklick auf Zeile 4 — Auswahl wird 4.

Bei mehreren Einträgen fragt Delete einmal: `Delete 2 items?`; bei einem Eintrag `Delete file '<name>'?` bzw. `Delete folder '<name>'?`.

7.6 Sprache

Seit `cufileops` 0.4 und `cuundo` 0.3 laufen Menüeinträge, Rückfragen und Fehlermeldungen über msgcat, sobald es geladen ist. Seit ctrlutils 0.4 auch die Gründe von cuundo und die Filterzeile von cufilter. Die Module laden msgcat selbst, wenn vorhanden. Texte mit Detail nehmen `%s`. Die Übersetzung legt die App im Namensraum `::ctrlutils` ab; msgcat sucht von `::ctrlutils::cufileops` aus auch dort:

```
package require msgcat
namespace eval ::ctrlutils {
    ::msgcat::mcset {} Rename Umbenennen
    ::msgcat::mcset {} "Could not rename:\n%s" "Umbenennen fehlgeschlagen:\n%s"
    ::msgcat::mcset {} "Delete file '%s'?" "Datei '%s' löschen?"
}
```

Ohne Katalog bleiben die Texte englisch. Die Form mit `{}` (gilt für jede Sprache) prüft der Test `cufileops-10.2`. Geprüft (Tcl/Tk 8.6.14, Linux, Xvfb): das Beispiel oben wörtlich, Locale `c` — Menüeintrag `Umbenennen` und Rückfrage `Datei 'a.txt' löschen?` kommen deutsch; mit `mclocale de` und `mcset de` ebenso. Die Schlüssel sind die englischen Texte selbst, etwa `New Folder`, `Open with %s`, `Open with...`, `(read-only)`, `Delete %s items?`, `Undo failed:\n%s`. Seit ctrlutils 0.4 kommen dazu `Filter:` und `x` aus cufilter und die Gründe von cuundo, etwa `no saved content to restore` oder `cannot undo op %s` (Test `cuundo-11.1`). In ctrlutils 0.3 bleiben diese englisch.

7.7 cuundo

cuundo zeichnet **nur** auf, was cufileops über `-onop` meldet. Die beiden kennen sich nicht; die Verbindung macht die App:

```
::ctrlutils::cuundo::install $f $prov -list $list
::ctrlutils::cufileops::install $f $prov $list $refresh \
    -onop [list ::ctrlutils::cuundo::record $f]
```

`cuundo::install` gibt `$f` zurück und leert beide Stapel. Ohne `-onop` bleibt der Undo-Stapel leer. Ein `record` vor `cuundo::install` für dieses `$f` geht still verloren. Ohne `-list` gibt es keine Tasten; `-list` bindet Strg+Z (undo) sowie Strg+Y und Strg+Umschalt+Z (redo) auf den Listenkörper. Auch hier ist eine unbekannte Option ein Fehler (`{CTRLUTILS CUUNDO OPTION <opt>}`). Nach Undo und Redo frischt cuundo die Liste auf: mit `-refresh skript`, sonst über `cufileops::refresh $f`, also den `$refreshcmd` von cufileops für dasselbe `$f`. Scheitert das Auffrischen, endet Undo oder Redo mit `{CTRLUTILS CUUNDO REFRESH}`; der Schritt selbst ist dann schon getan. Bis `cuundo` 0.4 rief cuundo die interne Prozedur `cufileops::_refresh` in `catch` auf, und ein Fehler blieb unsichtbar.

Operation	**Undo**
Rename von → nach	zurück verschieben

Operation	Undo
New Folder	löschen
Paste	Ziel löschen
Delete	aus Schnappschuss wiederherstellen
Cut + Paste	Ziel löschen, Quelle wiederherstellen

Den Schnappschuss nimmt cufileops **vor** dem Löschen (Bytes einer Datei oder ein ganzer Teilbaum, im Speicher). Rückgängig geht ein Löschen also nur, wenn der Provider den Inhalt lesen kann. Undo von New Folder und Paste hält seinerseits einen Schnappschuss fest, damit Redo das Gelöschte wiederherstellt.

Geprüft (Tcl/Tk 8.6.14, Linux, Xvfb): New Folder → Undo → Redo; zweites Paste rückgängig; Delete von `b.txt` rückgängig (Inhalt wieder da); Rename `b.txt` → `b2.txt` rückgängig; Cut von einem Tab in den anderen rückgängig — alles wie in der Tabelle.

Weitere Aufrufe: `undo`, `redo`, `canUndo` / `canRedo` (für Menüzustände), `clear`. Neue Aktivität leert den Redo-Stapel.

Fehler zeigt cuundo per `tkudialog::showError` an, falls tkudialog geladen ist (`Undo failed:` bzw. `Redo failed:` plus Grund). Ein gescheitertes Undo bleibt auf dem Undo-Stapel. Seit `cuundo` 0.4 (ctrlutils 0.4) bleibt auch ein gescheitertes Redo auf dem Redo-Stapel. Geprüft (Tcl 8.6.14 und 9.0.4, Linux, Xvfb): Test `cuundo-1.10` liefert danach `canRedo` 1; `cuundo.test` 13 von 13 bestanden. In `cuundo` 0.3 aus ctrlutils 0.3 ist ein gescheitertes Redo verloren: es landet auf keinem Stapel mehr (danach `canRedo` 0 und `canUndo` 0).

`cuundo` 0.4 verlangt `tclutils::tuprovider` nicht mehr; es ruft nur das Provider-Objekt auf, das es bei `install` bekommt.

Apps sollten auch eigene Änderungen (etwa ein Umbenennen aus einer Maske) als Operationssatz an `cuundo::record` geben — dieselbe Undo-Spur wie im Menü (Dateiblick: laut Review erledigt, hier nicht geprüft). Die Sätze, die cufileops erzeugt:

```
op mkdir   path P
op rename  from A to B type file|dir
op paste   path P type file|dir
op delete  path P type file|dir saved SNAP
op movex   srcprov S dstprov D from A to B type file|dir saved SNAP
```

SNAP ist `type file bytes …` oder `type dir children {name SNAP …}`. `movex` entsteht beim Cut; scheitert dort das Löschen der Quelle, meldet cufileops stattdessen `paste`.

7.8 Alles zusammen

Lauffähig unter `wish` (Tablelist auf dem `auto_path`):

```
source /pfad/zu/tclutils/tools/setup.tcl
tcl::tm::path add /pfad/zu/ctrlutils/lib/tm
package require Tk
package require ctrlutils
package require tclutils::tuprovider
package require tkutils::tkufilelist
```

```
set prov [::tclutils::tuprovider open local]
set f    [ttk::frame .tab]
::tkutils::tkufilelist::widget $f.l -provider $prov -dir [pwd]
::ctrlutils::cufilter::widget $f.filter $f.l
pack $f.filter -side top -fill x
pack $f.l -side top -fill both -expand 1
pack $f -fill both -expand 1

set refresh [list ::tkutils::tkufilelist::refresh $f.l]
::ctrlutils::cuundo::install $f $prov -list $f.l
::ctrlutils::cufileops::install $f $prov $f.l $refresh \
    -onop [list ::ctrlutils::cuundo::record $f]
::ctrlutils::cumonitor::install $f $prov -interval 2000 \
    -onchange [list apply {{cmd dir} {uplevel #0 $cmd}} $refresh]
::ctrlutils::cumonitor::watch $f [pwd]
```

Geprüft (Tcl/Tk 8.6.14 und 9.0.4, Linux, Xvfb, tclutils 0.69.0): das Beispiel läuft; die Bindungen am Listenkörper sind danach `Button-3`, Entf, F2, Strg+C/V/X/Y/Z und Strg+Umschalt+Z.

Der Explorer verdrahtet genauso, mit einem eigenen Refresh-Callback, der nach jeder Operation auch den Baum auffrischt und `cumonitor::poke` aufruft.

7.9 Zusammenfassung

- caps steuern das Menü; "(read-only)" nur, wenn gar nichts passt.
- cufileops 0.3 = Mehrfachauswahl beim Rechtsklick + Dialog 0.3; 0.4 = Texte über msgcat; 0.5 = `tkufilelist` geladen, `owSetApp`; 0.6 = Cut nach Teilfehler bleibt, `refresh`.
- Eigene Apps setzen `cufileops::owSetApp meineapp`, sonst teilen sie das "Open with"-Register mit dem Explorer.
- Undo nur mit `-onop [list ::ctrlutils::cuundo::record $f]`; Tasten nur mit `cuundo::install … -list $list`.
- Ein gescheitertes Undo oder Redo bleibt seit ctrlutils 0.4 auf seinem Stapel; in 0.3 ging ein gescheitertes Redo verloren.

Nächstes Kapitel: Explorer und Dateiblick

8 Explorer und Dateiblick

8.1 Zwei Rollen

App	Ort	Rolle
Explorer	ctrlutils/apps/explorer/	Referenz / Demo für Controller
Dateiblick	dateiblick/	Endbenutzer-App

Namen und Bäume getrennt lassen. Explorer in Reviews und Regeln **unangetastet**.

Der Explorer ist die einzige App in den drei Repos, die ctrlutils benutzt. Die Apps unter `tkutils/apps/` (Editoren, Launcher, tkdevtools …) bauen nur auf tclutils und tkutils; die unter `tclutils/apps/` (`find-tclconfig`, `build-app`) sind Tk-frei.

8.2 Explorer (Kurz)

```
wish ctrlutils/apps/explorer/explorer.tcl ?verzeichnis?
wish ctrlutils/apps/explorer/explorer.tcl -zip archiv.zip
wish ctrlutils/apps/explorer/explorer.tcl -remote URL ?provider-optionen?
wish ctrlutils/apps/explorer/explorer.tcl --check
```

- Ohne Argument startet der Explorer im aktuellen Verzeichnis.
- `-remote` nimmt `sftp://`, `ftp://`, `dav://`, `davs://` oder `http(s)://` und lädt den passenden Provider erst bei Bedarf.
- `--check` (oder `-check`) zeigt, welche optionalen Vorschauen geladen sind (Bildformate über `img::*`, SVG über `tcllunasvg`, PDF über `pdfiumtcl`, SQLite) — auf der Konsole und in einem Fenster. Nach dem Schließen des Fensters endet er mit Exit 0 nur, wenn alle da sind. Geprüft (Tcl/Tk 8.6.14, Linux, Xvfb): hier fehlen `img::jpeg`, `img::tiff`, `img::bmp`, `img::ico`, `tcllunasvg` und `pdfiumtcl`; SQLite ist `OK`.

Die drei Bibliotheken müssen auf dem Modulpfad liegen (Setup oder `site-tcl`, siehe Pairing und Setup). Der Explorer sourct `setup.tcl` nicht selbst. Dazu braucht er Tablelist (mit Scrollutil) aus tklib. Fehlt ein Pflichtpaket, zeigt er ein Meldungsfenster "Explorer cannot start" statt eines leeren Fensters und endet mit Exit 1. Die Meldung nennt das Paket, die Fehlermeldung von `package require`, den durchsuchten Modulpfad und den `auto_path`:

```
Missing package:
  tkutils::tkufiletree

can't find package tkutils::tkufiletree

Searched module path (tcl::tm::path):
/usr/lib/tcltk/tcl8/site-tcl
...

Searched auto_path:
...
```

Geprüft (Tcl/Tk 8.6.14 und 9.0.4, Linux, Xvfb) ohne Modulpfad zu den drei Bibliotheken. Die Pflichtmodule sind `.tm`-Module und werden über den Modulpfad gefunden, nicht über `auto_path`. Bis ctrlutils 0.3 nannte die Meldung nur den `auto_path`.

Je Tab: `tkufiletree`, `tkufilelist` mit `cufilter`-Zeile darüber, `tkupreview`; dazu `cuundo::install … -list`, `cufileops::install … -onop [list ::ctrlutils::cuundo::record $f]` und `cumonitor` mit 2000 ms.

Jeder Tab hält seinen **eigenen** Provider. Ein Tab kann ein ZIP zeigen, ein anderer die lokale Platte; cufileops kopiert zwischen ihnen, weil seine Zwischenablage den Quell-Provider kennt. Der Knopf "+" und Strg+T öffnen einen neuen lokalen Tab im Home-Verzeichnis. Beim Schließen eines Tabs stoppt der Explorer dessen `cumonitor` und zerstört den Provider, wenn kein anderer Tab ihn noch benutzt. Geprüft (Tcl/Tk 8.6.14, Linux, Xvfb): Start mit `-zip` ergibt einen Tab mit Provider `::tclutils::tuprovider::Zip`, ein neuer Tab bekommt ein eigenes Provider-Objekt.

`cusync` ist optional geladen: fehlt es, startet der Explorer trotzdem; erst "File > Sync folders..." meldet `Sync needs the ctrlutils::cusync module.` und rät, das Modul neben die übrigen ctrlutils-Module zu legen. Ebenso optional ist `tkutils::tkutlfind` für die Suche in der Liste; fehlt es, sagt die Meldung, das Modul zu den übrigen tkutils-Modulen zu legen. Bis

ctrlutils 0.4 nannten beide Meldungen einen Dateinamen (`cusync-0.2.tm`, `tkutlfind-0.2.tm`); die Datei heißt seit tkutils 0.49.0 aber `tkutlfind-0.3.tm`. Das Menü "Edit" bietet Undo, Redo und dieselben Dateioperationen wie das Kontextmenü. Fehler beim Start und Hintergrundfehler zeigt `tkudialog` mit kopierbaren Details. Die Oberflächentexte des Explorers selbst sind englisch und laufen nicht über msgcat.

Als Einzelprogramm lässt sich der Explorer mit `build-app` aus tclutils bauen (Tcl-9-Zipkit mit allen drei `lib/tm`-Bäumen plus Tablelist und Scrollutil). Das beschreibt `tclutils/apps/build-app/doc/`; hier nicht geprüft.

8.3 Dateiblick (Kurz)

Dateiblick liegt nicht in den drei Repos. Die folgenden Punkte stammen aus dem Review-Stand und sind für diesen Stand **nicht geprüft**:

- Handverdrahtete GUI (kein `tkuload::slot` / `.tkd`)
- Requires u. a. `tkudialog` 0.3, `cufileops` 0.3
- Suite gegen Geschwister-Utils grün (Stand Reviews: 45/45)
- Workarounds für Multi-Select und Dialog-Defaults entfernt — Libs tragen das

Architektur-Notiz: `00review/dateiblick-review.md` (im Dateiblick-Baum).

`package require ctrlutils::cufileops 0.3` lädt auch 0.5 (gleiche Hauptversion). Wer Dateiblick auf Deutsch zeigen will, legt die Übersetzungen im Namensraum `::ctrlutils` ab (siehe cufileops und Undo) und für die Dialoge in `::tkutils`. Das Register für "Open with" teilt sich Dateiblick mit dem Explorer, bis es `::ctrlutils::cufileops::owSetApp dateiblick` aufruft (ab ctrlutils 0.4).

8.4 Zusammenfassung

- Explorer = Referenz, Dateiblick = Produkt.
- Beide auf demselben Stack; nicht vermischen.
- Im Explorer hat jeder Tab seinen eigenen Provider.

Nächstes Kapitel: Checkliste App

9 Checkliste App

---

1. Pairing **0.69.0** / **0.49.0** / **0.4** laden; im Banner der Suite prüfen, welche Dateien wirklich geladen werden.
2. `setup.tcl` (oder `TCLUTILS_TM` / `TKUTILS_TM`), dazu `tcl::tm::path add …/ctrlutils/lib/tm` — oder alles in `site-tcl`, alte Moduldateien vorher löschen.
3. `package require Tk` vor `ctrlutils`. Bis ctrlutils 0.3 `tkutils::tkufilelist` selbst laden; ab 0.4 lädt `cufileops` es.
4. Provider wählen; caps prüfen, bevor Menüs hart verdrahtet werden.
5. Widgets: `tkufilelist` / Tree / Path; Dialoge **0.3**; tablelist **0.4**.
6. `cuundo::install $f $prov -list $list`, dann `cufileops::install … -onop [list ::ctrlutils::cuundo::record $f]` — beide mit demselben `$f`; optional `cufilter::widget` (selbst packen) und `cumonitor`.

7. Refresh-Callback nach Mutation, darin `cumonitor::poke`; beim Tab-Ende `cumonitor::stop` und Provider `destroy`.
8. Eigene Dateiänderungen der App als Operationssatz an `cuundo::record` geben.
9. Sprache: Übersetzungen für cufileops, cuundo und cufilter per `::msgcat::mcset` im Namensraum `::ctrlutils`, für tkudialog in `::tkutils`.
10. Eigenen App-Namen für das "Open with"-Register setzen, beim Start: `::ctrlutils::cufileops::owSetApp meineapp` (ab ctrlutils 0.4).
11. Settings/Geometrie: `tusettings` / `tkugeometry`.
12. Tests: `tclsh tests/all.tcl` über tutestrun; Banner prüfen, Grenze über *_TESTGRENZE; Tk-Tests mit Display (`xvfb-run -a`).
13. Kalender: tical 1.1.1 über `auto_path` laden (nicht `tcl::tm::path`), Feiertage mit `-holidays DE`, Termine mit `-events` im Format von `tical::event`; als Widget `tkutils::tkutical` oder `tkutils::tkumonthcanvas` einzeln laden.
14. Explorer nicht anfassen; eigene App im eigenen Baum.

9.1 Zusammenfassung

- Stack laden → Provider → View → Controller → Lebenszyklus.
- Details: Bücher *tclutils* und *tkutils*.

Nächstes Kapitel: Kalender: tical

10 Kalender: tical

10.1 Was tical ist

tical 1.1.1 ist eine Kalender- und iCalendar-Engine in reinem Tcl, mit eigenem Repository neben tclutils, tkutils und ctrlutils. Sie trennt drei Schichten:

```
tical::core        Kalenderdaten eines Monats (Tage, ISO-Wochen, heute)
tical::view::*     ViewSpec: Raster für Monat, Woche oder Tag
tical::render::*   Ausgabe: Text, Tk-Canvas, PDF
```

Ein ViewSpec ist ein Dict mit einem Raster aus Zellen; jeder Renderer zeichnet jeden ViewSpec seines Typs. Termine haben ein gemeinsames Format (`tical::event`); die Views verteilen sie auf die Zellen. Dazu kommen Feiertage (`tical::holidays`), iCalendar-Export (`tical::io::ical`) und -Import (`tical::io::import`), ein Terminmodell im Speicher (`tical::model`) und ein SQLite-Adapter (`tical::adapter::sqlite`).

Der Kern braucht kein Tk und läuft unter Tcl 8.6 und 9. Vier Schichten sind optional und brauchen je ein Fremdpaket:

Paket	braucht
tical::render::canvas	Tk
tical::render::pdf	pdf4tcl
tical::adapter::sqlite	tdbc::sqlite3
tical::io::import	tclutils: tclutils::tuical

Das Sammelpaket `tical` lädt alles andere: `core`, `util`, `config`, `locale`, `term`, `holidays`, `holidays::de`, `event`, `model`, `io::ical`, die drei Views und `render::term`. Die vier optionalen Pakete lädt die Anwendung einzeln.

10.2 Laden

tical bringt ein `pkgIndex.tcl` im Wurzelordner mit; es lädt jede Datei mit `source -encoding utf-8`, auch unter Tcl 8.6 mit cp1252. Die Dateien heißen zwar `*.tm`, liegen aber in Unterordnern (`core/`, `views/`, `render/` …) und werden über `auto_path` gefunden, nicht über den Modulpfad:

```
lappend auto_path /pfad/zu/tical
package require tical                    ;# -> 1.1
package require tical::render::canvas    ;# nur mit Tk
```

`tical::io::import` findet tclutils über den Modulpfad, wie jede Anwendung des Stacks:

```
tcl::tm::path add /pfad/zu/tclutils/lib/tm
package require tical::io::import        ;# -> 1.1
```

Mit `tcl::tm::path add /pfad/zu/tical` findet Tcl nur die Datei des Sammelpakets; danach endet `package require tical` mit `can't find package tical::util 1.0`. Statt `lappend auto_path` geht auch `TCLLIBPATH` oder ein Ordner, der schon im `auto_path` steht (Kapitel Pairing und Setup).

tkutils sucht tical für seine Kalender-Widgets an zwei Stellen: in `TICAL_DIR` und, ohne diese Variable, in Geschwisterordnern `tical*` neben dem tkutils-Checkout. So machen es die Tests und `bin/tkutical.tcl`. Eine eigene Anwendung ergänzt `auto_path` selbst.

10.3 Verhältnis zu tclutils und tkutils

tclutils und ctrlutils laden tical nie. tkutils lädt es nur in zwei optionalen Widgets, die nicht im Sammelpaket `tkutils` stehen. Für viele Kalenderaufgaben gibt es in tclutils oder tkutils ein eigenes Modul ohne tical:

Aufgabe	tclutils	tkutils	tical
Monat als Text	tucal (wie cal)	tkucal (zeigt tucal)	render::term
Monat als Bild oder PDF	tumonthpng (PNG)	—	render::pdf (PDF)
Kalender-Widget	—	tkucalendar, tkudateentry (nur Tk und clock); tkutical, tkumonthcanvas (auf tical)	render::canvas
Feiertage	tuholiday	—	holidays, holidays::de
Wiederholregeln expandieren	turrule	—	event::expand (eigener Code, dieselben Termine)

Aufgabe	tclutils	tkutils	tical
iCalendar lesen	tuical::parse	tkuical (Tabelle)	io::import::parse (über tuical)
iCalendar schreiben	tuical::toIcs	—	io::ical::exportEvents

Die beiden tkutils-Widgets auf tical:

- `tkutils::tkutical` 0.4: Monats- oder Wochenansicht mit Navigation, Wochennummern und Tagesauswahl; zeichnet über `tical::render::canvas`; Termine mit `-events` (seit 0.4, braucht tical 1.1), Serien an jedem ihrer Tage.
- `tkutils::tkumonthcanvas` 0.5: Monat, Quartal oder Jahr auf einem eigenen Canvas mit Themes und Notizen; nimmt Datum, Namen und Feiertage aus `tical::config`, `tical::locale`, `tical::holidays` und `tical::view::month`. Die Feiertagsregion leitet es aus der Locale ab (`de_DE` → `DE`).

Feiertage: tuholiday aus tclutils liefert die bundesweiten Tage einschließlich Oster- und Pfingstsonntag (11 im Jahr 2026), `tical::holidays::de` die neun gesetzlichen Tage ohne die beiden Sonntage; mit `-sundays 1` dieselbe Liste wie `tuholiday` (siehe "Feiertage"). Beide rechnen Ostern selbst. Länderfeiertage kennt keines.

turrule expandiert eine RRULE in Termine, tuical liest iCalendar-Text. tical hat keinen eigenen Leser: `tical::io::import` ruft `tuical` auf und liefert Termine im Format von `tical::event` (Abschnitt Import). Serien rechnet tical seit 1.1.1 selbst aus (`tical::event::expand`), damit die Views sie auch ohne tclutils zeigen; ein Test vergleicht das Ergebnis mit `turrule`.

10.4 Views

Alle Views liefern einen ViewSpec. Fehlende Optionen kommen aus `tical::config`:

```
tical::config::getDefaults
;# -> firstweek iso locale de_DE today {} weekstart mon theme default timezone
    Europe/Berlin
```

`tical::config::set` prüft seit 1.0.2 Schlüssel und Wert: ein Tippfehler wie `weekStart` endet mit Errorcode `TICAL CONFIG KEY`, eine Zone, die Tcl nicht kennt, eine Locale außer Deutsch, Englisch, Französisch oder `weekstart sonntag` mit `TICAL CONFIG VALUE`; der alte Wert bleibt. `today` (`YYYY-MM-DD`, leer: das echte Datum) legt fest, welcher Tag als heute markiert wird, etwa für gleichbleibende Ausgaben.

View	type	Optionen	Zellen
tical::view::month::getData	month-grid	-year, -month (Pflicht), -tz, -weekStart mon|sun, -firstWeek iso|simple|us, -showAdjacentDays 0|1 (1), -holidays REGION, -count N (1), -events LISTE	42 (6 × 7)
tical::view::week::getData	week-grid	-year und -week oder -date, -tz, -holidays, -events	7

View	type	Optionen	Zellen
tical::view::day::getData	day-grid	-date (Pflicht), -hourStart (0), -hourEnd (23), -tz, -holidays, -events	eine je Stunde

`-date` nimmt `2025-10-14` und `20251014`. Seit 1.1.1 prüfen die Views auch die Werte: `-month` 1 bis 12, `-week` 1 bis 52 oder 53 (die ISO-Wochen des Jahres; vorher rutschte `-week 60` still ins nächste Jahr), ein gültiges `-date`, `-hourStart` nicht nach `-hourEnd`. Ein falscher Wert endet mit Errorcode `TICAL VIEW OPTION`, nicht mehr mit einem `CLOCK …`-Fehler.

Eine unbekannte Option oder eine Option ohne Wert ist ein Fehler mit Errorcode `TICAL VIEW OPTION`; die Meldung nennt die bekannten Optionen. Bis 1.0.1 überging die View solche Optionen still, ein Tippfehler wie `-weekstart` blieb ohne Wirkung:

```
tical::view::month::getData -year 2025 -month 10 -weekstart sun
;# Fehler: view::month::getData: unknown option "-weekstart", must be -count,
;#   -events, -firstWeek, -holidays, -month, -showAdjacentDays, -tz,
;#   -weekStart or -year
```

Der Wochenstart ist eine Option der View, nicht der Renderer: Mit `-weekStart sun` (auch `sunday`) beginnt das Raster am Sonntag, und alle Renderer beschriften ihre Spalten nach den Zellen. Die Wochenansicht läuft immer von Montag bis Sonntag (ISO-Woche).

`-firstWeek` legt die Wochennummer der Zellen fest: `iso` (Vorgabe, ISO 8601: Woche 1 enthält den ersten Donnerstag), `simple` (Woche 1 sind der 1. bis 7. Januar) oder `us` (Wochen ab Sonntag, Woche 1 enthält den

1. Januar). Der 1. Januar 2023 war ein Sonntag:

```
lmap fw {iso simple us} {
    set m [tical::view::month::getData -year 2023 -month 1 \
        -weekStart sun -firstWeek $fw]
    dict get [lindex [dict get $m cells] 0] week
}
;# -> 52 01 01
```

Die Renderer zeigen je Rasterzeile die Woche der mittleren Zelle; so nennt auch ein Sonntagsraster mit ISO-Wochen die Woche, zu der sechs der sieben Tage gehören.

Eine Zelle ist ein Dict mit `date`, `dow` (1 = Montag), `week`, `markers` und `events`; im Monat dazu `inMonth`, in der Tagesansicht `hour` und `time`. `markers` enthält `today` und `holiday`, `events` die Termine (Abschnitt Termine). Mit `-showAdjacentDays 0` sind die Zellen vor und nach dem Monat leer (`date {}`).

```
set s [tical::view::month::getData -year 2025 -month 10 -holidays DE]
dict keys $s
;# -> type month year cols rows weekStart firstWeek cells legend grid
llength [dict get $s cells]                    ;# -> 42
set w [tical::view::week::getData -date 2025-10-14]
dict get $w week                               ;# -> 42
set d [tical::view::day::getData -date 2025-10-03 \
    -hourStart 8 -hourEnd 17 -holidays DE]
dict get $d rows                               ;# -> 10
dict get [lindex [dict get $d cells] 0] markers  ;# -> holiday
```

`-date` in der Wochenansicht rechnet mit dem ISO-Jahr: Der 30.12.2024 liegt in Woche 1 von 2025.

```
set w [tical::view::week::getData -date 2024-12-30]
list [dict get $w year] [dict get $w week]    ;# -> 2025 1
```

Die Region bei `-holidays` ist der Schlüssel, unter dem das Feiertags-Plugin registriert ist, etwa DE; Groß- und Kleinschreibung zählen nicht (de wirkt wie DE). Eine unbekannte Region markiert keinen Tag; einen Fehler gibt es nicht.

`-count N` mit N > 1 liefert statt eines ViewSpecs eine Liste von N aufeinanderfolgenden Monaten ab `-year`/`-month`, auch über den Jahreswechsel. `-count 1` (Vorgabe) liefert einen ViewSpec wie ohne die Option; kleinere Werte enden mit `view::month::getData: -count must be an integer >= 1, got "0"`.

```
set q [tical::view::month::getData -year 2025 -month 11 -count 3]
llength $q                                    ;# -> 3
lmap m $q {dict get $m month}                 ;# -> 11 12 1
```

Die Liste geht so, wie sie ist, an `render::pdf::createQuarter` oder `createYear`.

`validate` prüft in jeder View die Pflichtfelder und den `type`.

10.5 Termine

Ein Termin ist in ganz tical dasselbe Dict, angelehnt an iCalendar (tical::event):

Schlüssel	Bedeutung
summary	Titel (Pflicht, darf leer sein)
dtstart	Beginn (Pflicht): YYYYMMDD ganztägig, YYYYMMDDTHHMMSS lokal, YYYYMMDDTHHMMSSZ UTC
dtend	Ende (optional), von derselben Art; ganztägig exklusiv, also der Tag nach dem letzten Tag
allday	0 oder 1; folgt dtstart
uid, description, location, status, categories, rrule, exdate	optional

`tical::event::normalize` bringt einen Termin in diese Form und prüft ihn. Es nimmt auch ISO-Angaben (`2025-10-14`, `2025-10-14T09:00`) und für eine Version (1.1) die älteren Feldnamen: `title`, `date`, `startTime`, `endTime` aus `tical::model`, `starts_at` und `ends_at` aus `tical::adapter::sqlite`.

```
tical::event::normalize {summary Teamrunde dtstart 2025-10-14T09:00 dtend
    2025-10-14T10:00}
;# -> summary Teamrunde dtstart 20251014T090000 dtend 20251014T100000 allday 0
tical::event::normalize {summary Messe dtstart 2025-10-20 dtend 2025-10-23}
;# -> summary Messe dtstart 20251020 dtend 20251023 allday 1
tical::event::normalize {title Zahnarzt date 2025-10-14 startTime 09:00}
;# -> summary Zahnarzt dtstart 20251014T090000 allday 0
tical::event::days {summary Messe dtstart 20251020 dtend 20251023 allday 1}
;# -> 2025-10-20 2025-10-21 2025-10-22
```

Die Messe dauert drei Tage: `dtend` 20251023 ist exklusiv. Ein Ende vor dem Beginn endet mit `dtend "20251014T090000" is before dtstart "20251014T100000"` (Errorcode `TICAL EVENT RANGE`).

`-events LISTE` gibt einer View die Termine. Sie legt jeden Termin in das `events` jeder Zelle, die er berührt, auch in Nachbartage des Monats. Ein Eintrag ist der Termin plus `part` (`single`, `start`, `middle`, `end`) und `time` (Beginn `HH:MM` am ersten Tag, sonst leer); ganztägige Termine stehen vor den anderen. Eine Serie (`rrule`) steht an jedem ihrer Tage, mit `recurrence-id`; bis 1.1 stand sie nur am ersten. `tical::event::expand termine von bis ?tz?` liefert die Vorkommen auch direkt. Es kennt `FREQ` (täglich bis jährlich), `INTERVAL`, `COUNT`, `UNTIL`, `BYDAY` (auch `1MO`, `-1FR`), `BYMONTH`, `BYMONTHDAY` und `WKST`, beachtet `EXDATE` und Ausnahmen mit `RECURRENCE-ID`.

```
set termine {
    {summary Teamrunde dtstart 2025-10-14T09:00 dtend 2025-10-14T10:00}
    {summary Messe dtstart 2025-10-20 dtend 2025-10-23}
}
set s [tical::view::month::getData -year 2025 -month 10 -events $termine]
set r {}
foreach c [dict get $s cells] {
    foreach e [dict get $c events] {
        lappend r [list [dict get $c date] [dict get $e summary] \
            [dict get $e part] [dict get $e time]]
    }
}
set r
;# -> {2025-10-14 Teamrunde single 09:00} {2025-10-20 Messe start {}}
;#    {2025-10-21 Messe middle {}} {2025-10-22 Messe end {}}
```

Die Tagesansicht hat zusätzlich einen Schlüssel `events` mit allen Terminen des Tages; ihre Stundenzellen enthalten die Termine mit Uhrzeit, die in dieser Stunde laufen:

```
set t [tical::view::day::getData -date 2025-10-14 \
    -hourStart 8 -hourEnd 10 -events $termine]
lmap c [dict get $t cells] {lmap e [dict get $c events] {dict get $e summary}}
;# -> {} Teamrunde {}
```

Ein Termin mit `Z` wird in die Zeitzone der View (`-tz`) umgerechnet; ein Termin ohne `Z` gilt dort als lokale Zeit.

10.6 Renderer

10.6.1 Text: render::term

```
tical::render::term::print $viewSpec ?-weekNumbers 0|1? ?-color 0|1? ?-weekStart
    mon|sun?
```

Gibt den Monat als Text zurück, sechs Zeilen zu sieben Tagen; Tage aus dem Nachbarmonat stehen als `...` Woche und Tag gibt es seit 1.1.1 als Liste (siehe unten); ein anderer Typ endet mit `render::term::print: expected type month-grid, week-grid or day-grid, got "year-grid"`. Jede Spalte ist vier Zeichen breit, auch die Wochenspalte und die Kopfzeile; die Zahlen stehen rechtsbündig untereinander.

Die Zellen, jeweils vier Zeichen zwischen den Anführungszeichen:

```
" 17 "          Tag
"(18)"  " (3)"  Feiertag der View (-holidays DE)
"[17]"  " [3]"  heute, nur mit -color 1; heute geht vor Feiertag
" 17*"          Tag mit Termin (events der Zelle)
"[17*"  "(18*"  heute oder Feiertag mit Termin
" .. "          Tag aus dem Nachbarmonat
```

Bis 1.0.1 stand ein Feiertag als `+18`, heute als `*17`; `+18` las sich wie eine Rechnung.

Monats- und Tagesnamen kommen aus tical::locale und folgen `tical::config::set locale …`; mit der Vorgabe `de_DE` heißt die Kopfzeile `Oktober 2025`, die Spaltenköpfe sind `Mo Di Mi Do Fr Sa So` (mit `-weekNumbers 1` davor `KW`). Nach `tical::config::set locale en_US` steht dort `October 2025` und `Wk Mo Tu We Th Fr Sa Su`.

Die Spaltenfolge kommt aus den Zellen der View. `-weekStart` nimmt `render::term` nur als Prüfung an: Stimmt der Wert mit der View überein, bleibt die Ausgabe gleich; widerspricht er, endet der Aufruf mit Errorcode `TICAL RENDER WEEKSTART`. Code, der den passenden Wert übergibt, läuft so weiter. Unbekannte Optionen enden mit `render::term::print: unknown option "-colour", must be -color, -weekNumbers or -weekStart`.

```
set so [tical::view::month::getData -year 2025 -month 10 \
    -weekStart sun -holidays DE]
puts [tical::render::term::print $so]
string equal [tical::render::term::print $so -weekStart sun] \
    [tical::render::term::print $so]          ;# -> 1
tical::render::term::print $so -weekStart mon
;# Fehler: render::term::print: -weekStart mon contradicts the view (starts on
;#   sun): week start comes from the view; use getData ... -weekStart mon
```

```
 Oktober 2025
 So  Mo  Di  Mi  Do  Fr  Sa
 ..  ..  ..   1   2  (3)  4
  5   6   7   8   9  10  11
 12  13  14  15  16  17  18
 19  20  21  22  23  24  25
 26  27  28  29  30  31  ..
 ..  ..  ..  ..  ..  ..  ..
```

Dieselben Namen liefert `tical::locale` direkt:

```
tical::locale::getMonthName 10               ;# -> Oktober
tical::locale::getWeekdaysShort              ;# -> Mo Di Mi Do Fr Sa So
tical::locale::getWeekdaysShort 7            ;# -> So Mo Di Mi Do Fr Sa
tical::locale::getWeekLabel short            ;# -> KW
```

`tical::locale` kennt `de`, `en` und `fr` und folgt `tical::config::set locale …`. Das optionale Argument der Wochentagslisten ist der erste Tag (1 = Montag … 7 = Sonntag); `getWeekLabel` liefert `KW`, `Week` oder `Semaine`, mit `short` `KW`, `Wk` oder `Sem`.

Woche und Tag als Liste. Die Woche hat eine Zeile je Tag, die Termine stehen hinter dem Tag, einer je Zeile: `HH:MM Titel` mit Uhrzeit, ganztägige ohne, `.. Titel` für einen Termin, der an einem früheren Tag begonnen hat. Ein Feiertag steht in runden Klammern, heute (mit `-color` 1) in eckigen.

```
set termine {
    {summary Zahnarzt dtstart 2025-10-13T09:00 dtend 2025-10-13T10:00}
    {summary Training dtstart 2025-10-13T18:00 dtend 2025-10-13T19:30}
    {summary Nacht dtstart 2025-10-14T22:00 dtend 2025-10-15T02:00}
    {summary Urlaub dtstart 2025-10-16 dtend 2025-10-18}
}
puts [tical::render::term::print \
    [tical::view::week::getData -date 2025-10-14 -events $termine]]
```

```
 KW 42 2025
 Mo 13 Okt   09:00 Zahnarzt
             18:00 Training
 Di 14 Okt   22:00 Nacht
 Mi 15 Okt   .. Nacht
 Do 16 Okt   Urlaub
 Fr 17 Okt   .. Urlaub
 Sa 18 Okt
 So 19 Okt
```

Der Tag hat eine Zeile je Stunde der View; ganztägige Termine stehen unter dem Titel, ein Termin, der vor der ersten Stunde begann, in der ersten Zeile mit `..`:

```
puts [tical::render::term::print [tical::view::day::getData \
    -date 2025-10-15 -hourStart 0 -hourEnd 2 -events $termine]]
```

```
 Mi 15 Okt 2025
00:00 | .. Nacht
01:00 |
02:00 |
```

`-weekNumbers` wirkt nur im Monat (die Woche nennt ihre Nummer im Titel), `-weekStart` prüft Monat und Woche; die Woche beginnt immer am Montag.

10.6.2 Tk: render::canvas

```
tical::render::canvas::draw $canvas $viewSpec ?-interactive 0|1? ?-fontsize 12?
    ?-weekNumbers 0|1?
```

Zeichnet Monat, Woche oder Tag auf einen vorhandenen Canvas: heute gelb, Feiertage rot, Wochenende blau, Nachbartage grau. Termine der Zellen erscheinen im Monat als bis zu drei Punkte unter der Zahl, in Woche und Tag als Titel mit Uhrzeit (Tag `event`); die Spalten der Woche werden dafür breiter. Unbekannte Optionen enden mit Errorcode `TICAL RENDER OPTION`. Monatsname, Wochentage und der Titel der Wochenansicht (`KW 42, 2025`) kommen aus `tical::locale`, die Spaltenfolge aus der View. Mit `-interactive 1` reagieren die Tage in Monat und Woche auf Maus und Klick; die Tagesansicht hat keine Bindungen. Die Auswahl gilt je Canvas:

Aufruf	Wirkung
setSelectMode $c none|single|multiple	Auswahlart; Vorgabe none
getSelection $c	sortierte ISO-Daten
setSelection $c {daten}	Einzeldaten und Bereiche A..B
clearSelection $c	leert die Auswahl
setSelectionCommand $c cmd	cmd $c $auswahl nach jeder Änderung
setCallback ?$c? cmd	nur bei none: cmd $c $datum beim Klick; ohne $c für alle Canvas, mit $c nur für diesen (geht vor)

Bei `multiple` schaltet ein Klick den Tag um, ein Klick mit Umschalt wählt den Bereich vom zuletzt geklickten Tag bis hier.

```
canvas .c -width 400 -height 350 -bg white
pack .c
tical::render::canvas::draw .c $s -interactive 1 -weekNumbers 1
tical::render::canvas::setSelectMode .c multiple
tical::render::canvas::setSelection .c {2025-10-06..2025-10-08 2025-10-20}
;# -> 2025-10-06 2025-10-07 2025-10-08 2025-10-20
```

`setCallback` nimmt ein Kommandopräfix, davor wahlweise den Canvas: `setCallback [list apply {{c datum} {puts $datum}}]` gilt für alle, `setCallback .c …` seit 1.1.1 nur für `.c`. Die Form mit Parameterliste und Rumpf als zwei Argumente endet mit Errorcode `TICAL RENDER ARGS` (`"c datum" is not a widget path`).

10.6.3 PDF: render::pdf

```
tical::render::pdf::create        datei $viewSpec ?optionen?
tical::render::pdf::createQuarter datei {spec spec spec} ?optionen?
tical::render::pdf::createYear    datei {12 specs} ?optionen?
tical::render::pdf::drawMonth|drawWeek|drawDay $pdf $viewSpec ?optionen?
```

`create` schreibt eine Seite mit Monat, Woche oder Tag. `createQuarter` setzt drei Monate nebeneinander, `createYear` zwölf in ein Raster 4 × 3, beide im Querformat. Optionen mit oder ohne Bindestrich: `paper` (`a4`, `a3`, `a5`, `letter`, in beliebiger Schreibung), `orientation` (`portrait`, bei Quartal und Jahr `landscape`), `fontsize` (10), `margin` (20), `weekNumbers` (1), `showGrid` (1) und die Farben `colorToday`, `colorHoliday`, `colorWeekend`, `colorEvent`, `colorEventText`. Andere Namen und Werte enden mit Errorcode `TICAL RENDER OPTION`; bis 1.0.1 wurde aus `-paper Letter` still A4. `draw…` zeichnet in ein schon offenes `pdf4tcl`-Objekt. Rückgabe ist der Dateiname. Namen und Spaltenfolge wie bei `render::canvas`.

Termine der Zellen färben den Tag und stehen als Titel unter der Zahl, so viele, wie in die Zelle passen (die letzte Zeile dann +N); in den kleinen Blöcken von Quartal und Jahr bleibt ein Punkt. Die Option `events` aus 1.0.1 (Liste von Daten, die nur eingefärbt werden) nimmt 1.1 noch an; sie entfällt mit 1.2.

```
package require tical::render::pdf
set specs [tical::view::month::getData -year 2025 -month 10 \
    -count 3 -holidays DE -events $termine]
tical::render::pdf::createQuarter q4.pdf $specs   ;# -> q4.pdf
tical::render::pdf::create okt.pdf [lindex $specs 0] -paper Letter
;# -> okt.pdf
```

10.7 Feiertage

`tical::holidays` verwaltet Plugins je Region. Ein Plugin ist ein Befehl, der für ein Jahr ein Dict `ISO-Datum → Name` liefert; `tical::holidays::de` registriert sich beim Laden als `DE`, mit den neun bundesweiten Feiertagen. `getHolidays -sundays 1 jahr` nimmt Oster- und Pfingstsonntag dazu und liefert dann dieselbe Liste wie `tclutils::tuholiday::holidays`. Für die Views registriert man das als Region, unter `DE` oder einem eigenen Namen:

```
tical::holidays::register DE {::tical::holidays::de::getHolidays -sundays 1}
tical::holidays::getHolidays DE 2026 4
;# -> 3 Karfreitag 5 Ostersonntag 6 Ostermontag
```

Weil ein Plugin nur ein Befehlspräfix ist, das am Ende das Jahr bekommt, passt auch `tuholiday` direkt: `tical::holidays::register DE ::tclutils::tuholiday::holidays`.

```
tical::holidays::getHolidays DE 2026 4      ;# -> 3 Karfreitag 6 Ostermontag
tical::holidays::register AT ::meineapp::feiertageAT
```

`getHolidays` liefert für einen Monat ein Dict `Tag → Name`; `getMarkers` dasselbe als Markerliste. Eine unbekannte Region ergibt ein leeres Dict.

10.8 iCalendar

`tical::io::ical` schreibt Termine nach RFC 5545: VCALENDAR mit VEVENT, Zeilen mit CRLF, Zeilen über 75 Byte (UTF-8) gefaltet. Ein Termin ist ein Dict mit `uid`, `dtstart`, `summary` (Pflicht), dazu wahlweise `dtend` oder `duration`, `description`, `location`, `status`, `categories`, `rrule` und `exdate`. Zeiten stehen in der Form `YYYYMMDDTHHMMSSZ` (UTC); ohne `Z` schreibt `io::ical` sie unverändert, sie gelten dann als lokale Zeit. `exportEvents` schickt jeden Termin vorher durch `tical::event::normalize`; ISO-Angaben und die älteren Feldnamen gehen also auch, eine fehlende `uid` wird erzeugt. Ein Termin mit `recurrence-id` (eine verschobene Instanz einer Serie) bekommt `RECURRENCE-ID`, mit `TZID` wie `DTSTART`; seit 1.1.1 kommt er beim Wiedereinlesen als Ausnahme zurück, nicht als zweiter Termin. `makeEvent` ergänzt `uid` und `dtstamp`:

```
set ev [tical::io::ical::makeEvent -uid team-1@example.org \
    -dtstart 20251014T090000Z -dtend 20251014T100000Z \
    -summary "Teamrunde" -rrule {freq WEEKLY count 4}]
set ics [tical::io::ical::exportEvents [list $ev]]
```

Ergebnis ohne die Zeile `DTSTAMP` (die Uhrzeit des Exports):

```
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//tical//tical 1.1.1//EN
CALSCALE:GREGORIAN
METHOD:PUBLISH
BEGIN:VEVENT
UID:team-1@example.org
DTSTART:20251014T090000Z
DTEND:20251014T100000Z
SUMMARY:Teamrunde
RRULE:FREQ=WEEKLY;COUNT=4
END:VEVENT
END:VCALENDAR
```

`rrule` ist ein Dict mit `freq` (`DAILY`, `WEEKLY`, `MONTHLY`, `YEARLY`), `count` oder `until`, `interval`, `byday`, `bymonth`, `bymonthday` und `wkst`, oder eine RRULE-Zeichenkette (`FREQ=WEEKLY;COUNT=4`). Andere Frequenzen enden mit `FREQ must be DAILY|WEEKLY|MONTHLY|YEARLY, got "HOURLY"`. `exportToFile $events datei` schreibt dasselbe in eine Datei (UTF-8, Zeilenende CRLF; bis 1.0 stand dort `\r\r\n`).

10.8.1 Ganztägige Termine

Ein Datum ohne Uhrzeit (`20251020` oder `2025-10-20`) ist ein ganztägiger Termin. `io::ical` schreibt ihn mit `VALUE=DATE`; `DTEND` ist wie in iCalendar exklusiv. Bis 1.0.1 schrieb es ein Datum ohne `VALUE=DATE`, was RFC 5545 nicht erlaubt.

```
set ganz [tical::io::ical::makeEvent -uid messe-1@example.org \
    -summary Messe -dtstart 2025-10-20 -dtend 2025-10-23]
set ics2 [tical::io::ical::exportEvents [list $ganz]]
```

Der VEVENT ohne `DTSTAMP`, eine Messe vom 20. bis 22. Oktober:

```
BEGIN:VEVENT
UID:messe-1@example.org
DTSTART;VALUE=DATE:20251020
DTEND;VALUE=DATE:20251023
SUMMARY:Messe
END:VEVENT
```

`validate` prüft die Pflichtfelder, das Format von `dtstart` und `dtend` (`YYYYMMDD`, `YYYYMMDDTHHMMSS` oder `YYYYMMDDTHHMMSSZ`, beide von derselben Art, das Ende nicht vor dem Beginn) und die `rrule`. `exportEvents` ruft es nicht selbst auf. ISO-Angaben lehnt `validate` ab; es prüft, was so geschrieben würde:

```
tical::io::ical::validate $ev                ;# -> 1
tical::io::ical::validate {uid u1 dtstart 20251014 dtend 20251015 summary S}
;# -> 1
tical::io::ical::validate {uid u1 dtstart 2025-10-14 summary S}
;# Fehler: dtstart format invalid: "2025-10-14" (expected YYYYMMDD,
;#   YYYYMMDDTHHMMSS or YYYYMMDDTHHMMSSZ)
```

10.9 Import

tical::io::import liest iCalendar-Text und expandiert Wiederholungen. Es ist optional und braucht

tclutils: `tuical` zerlegt den Text; die RRULE rechnet `tical::event::expand` aus. Ohne tclutils endet `package require tical::io::import` mit `can't find package tclutils::tuical 0.1`; der Rest von tical lädt weiter.

Aufruf	Ergebnis
parse $ics	Liste von Terminen im Format von tical::event
readFile datei	dasselbe aus einer Datei (UTF-8)
expand $termine von bis ?tz?	Vorkommen, die die Tage von bis bis berühren, nach Beginn sortiert
eventsInRange $ics von bis ?tz?	expand [parse $ics] von bis

`parse` liest `UID`, `SUMMARY`, `DESCRIPTION`, `LOCATION`, `STATUS`, `CATEGORIES`, `DTSTART` und `DTEND` (mit `VALUE=DATE` ganztägig), `DURATION` (wenn `DTEND` fehlt), `RRULE`, `EXDATE` und `RECURRENCE-ID`. `TZID` bleibt als Schlüssel `tzid` stehen, mit der Uhrzeit dieser Zone; `tical::event` und die Views rechnen sie in die Anzeigezone um (`-tz`, sonst `tical::config`). Eine Serie wird in ihrer eigenen Zone expandiert: 09:00 in New York bleibt 09:00 dort, in Berlin ist das in den Tagen zwischen den beiden Umstellungen 14:00 statt 15:00. Verstanden werden IANA-Namen, Präfix-IDs wie `/mozilla.org/…/Europe/Berlin` und gängige Windows-Namen wie `W. Europe Standard Time`; `tical::event::tzid name` liefert die Tcl-Zone oder `""`. Eine TZID, die Tcl nicht kennt (eigene `VTIMEZONE`), lässt die Uhrzeit als lokale Zeit stehen. Der Export schreibt `DTSTART;TZID=Europe/Berlin:…` zurück, ohne `VTIMEZONE`. `expand` lässt Tage aus `EXDATE` weg und ersetzt ein Vorkommen, für das ein VEVENT mit derselben `UID` und passender `RECURRENCE-ID` existiert; hat dieser `STATUS:CANCELLED` (so löschen Google und Outlook einen einzelnen Termin einer Serie), fällt das Vorkommen weg. `EXDATE;VALUE=DATE` streicht bei einer Serie mit Uhrzeit den ganzen Tag. Jedes Vorkommen trägt `recurrence-id`, aber kein `rrule` mehr.

Der Export von oben, wieder gelesen:

```
package require tical::io::import
set evs [tical::io::import::parse $ics]
llength $evs                                    ;# -> 1
lmap e [tical::io::import::expand $evs 2025-10-01 2025-12-31] {
    dict get $e dtstart
}
;# -> 20251014T090000Z 20251021T090000Z 20251028T090000Z 20251104T090000Z
set back [lindex [tical::io::import::parse $ics2] 0]
list [dict get $back dtstart] [dict get $back dtend] [dict get $back allday]
;# -> 20251020 20251023 1
```

Das Ergebnis von `expand` geht unverändert an `getData … -events`. Eine RRULE mit Teilen, die tical nicht kennt (`BYSETPOS`, `BYWEEKNO`, `BYHOUR` …), endet mit Errorcode `TICAL IMPORT RRULE` statt mit zu vielen Terminen.

10.10 Modell und SQLite

`tical::model` hält Termine (`createAppointment -summary -dtstart ?-dtend? …`) und Ereignisse (`createEvent -summary -dtstart …`) im Speicher, mit `get…`, `update…`, `delete…`, `list…`, `getAppointmentsByDate` und `toEvents`. Die Datensätze haben die

Felder von `tical::event`, dazu `id`, `type` und `created`. Für eine Version (1.1) nimmt das Modell noch die alten Optionen `-title`, `-date`, `-startTime`, `-endTime`, `-allDay`, `-category` an, und jeder Datensatz enthält noch die gleichnamigen Felder; mit 1.2 fallen sie weg. `-tzid` gibt die Zone lokaler Zeiten an (seit 1.1.1, vorher eine unbekannte Option); der Datensatz und `toEvents` behalten sie.

```
tical::model::createAppointment -summary Zahnarzt \
    -dtstart 2025-10-14T09:00 -dtend 2025-10-14T09:30  ;# -> appt-1
dict keys [tical::model::getAppointmentsByDate 2025-10-14]   ;# -> appt-1
tical::model::createAppointment -title Kontrolle -date 2025-10-15 \
    -startTime 10:00                                    ;# -> appt-2
dict get [tical::model::getAppointment appt-2] dtstart  ;# -> 20251015T100000
```

`toEvents` liefert alle Datensätze als Termine (`uid` = Kennung), fertig für `getData … -events`.

`tical::adapter::sqlite` speichert Termine über `tdbc::sqlite3` in einer Tabelle `events`: `connect`, `create`, `read`, `update`, `delete`, `exists`, `listRange`, `listAll`, `disconnect`. `create` und `update` nehmen das Format von `tical::event` (`dtstart`, `dtend`), `read` und `list…` liefern es mit `allday`. Die Felder `starts_at` und `ends_at` aus 1.0.1 nimmt 1.1 noch an und liefert sie zusätzlich; mit 1.2 fallen sie weg. Die Spalten der Tabelle bleiben. `listRange db von bis ?tz?` nimmt ISO-Grenzen; ein Datum als Ende schließt den ganzen Tag ein. Seit 1.1.1 vergleicht es Zeitpunkte: lokale Zeiten und Daten gelten in `tz` (sonst `tical::config`), UTC-Werte als UTC; vorher wurden sie als Text verglichen. Lokale Zeiten mit `tzid` gelten in dieser Zone. Kategorien mit Komma bleiben eine Kategorie. `tzid` steht in einer eigenen Spalte; `migrate` ergänzt sie in einer Tabelle aus 1.1. Vorher ging die Zone beim Speichern verloren.

```
package require tical::adapter::sqlite
set db [tical::adapter::sqlite::connect :memory:]
tical::adapter::sqlite::create $db [dict create uid u1 \
    dtstart 2025-10-14T09:00 summary Teamrunde]            ;# -> u1
set e [tical::adapter::sqlite::read $db u1]
list [dict get $e dtstart] [dict get $e starts_at]
;# -> 20251014T090000 20251014T090000
llength [tical::adapter::sqlite::listRange $db 2025-10-01 2025-10-31]  ;# -> 1
tical::adapter::sqlite::disconnect $db
```

10.11 Beispiel

Monat Oktober 2025 mit Feiertagen und Terminen, als Text und als Widget:

```
lappend auto_path /pfad/zu/tical
package require tical

set termine {
    {summary Teamrunde dtstart 2025-10-14T09:00 dtend 2025-10-14T10:00}
    {summary Messe dtstart 2025-10-20 dtend 2025-10-23}
}
set s [tical::view::month::getData -year 2025 -month 10 \
    -holidays DE -events $termine]
puts [tical::render::term::print $s -weekNumbers 1]
foreach c [dict get $s cells] {
    if {"holiday" in [dict get $c markers]} { puts [dict get $c date] }
}
```

Ausgabe:

```
 Oktober 2025
 KW  Mo  Di  Mi  Do  Fr  Sa  So
 40  ..  ..   1   2  (3)  4   5
 41   6   7   8   9  10  11  12
 42  13  14* 15  16  17  18  19
 43  20* 21* 22* 23  24  25  26
 44  27  28  29  30  31  ..  ..
 45  ..  ..  ..  ..  ..  ..  ..

2025-10-03
```

Legende: `(3)` Feiertag, `14*` Tag mit Termin; mit `-color 1` stünde der heutige Tag in eckigen Klammern.

Dasselbe als Widget aus tkutils, mit Mehrfachauswahl:

```
lappend auto_path /pfad/zu/tical
source /pfad/zu/tkutils/tools/setup.tcl
package require Tk
package require tkutils::tkutical                 ;# -> 0.4

set w [::tkutils::tkutical::widget .cal -year 2025 -month 10 \
    -holidays DE -selectmode multiple \
    -events {{summary Sport dtstart 2025-10-07T18:00 rrule {freq WEEKLY}}} \
    -command {apply {{w sel} {puts "Auswahl: $sel"}}}]
pack $w -fill both -expand 1
::tkutils::tkutical::next $w
::tkutils::tkutical::getMonth $w                  ;# -> 2025 11
::tkutils::tkutical::setSelection $w {2025-11-03..2025-11-05}
;# -> 2025-11-03 2025-11-04 2025-11-05
```

10.12 Fehler

Meldungen von tical sind englisch und tragen einen Errorcode der Form `TICAL <MODUL> <GRUND>`, wie `TCLUTILS <MODUL> <GRUND>` in tclutils. Module: `CORE`, `CONFIG`, `UTIL`, `LOCALE`, `EVENT`, `VIEW`, `RENDER`, `MODEL`, `ICAL`, `IMPORT`, `SQLITE`. Falsche Optionen tragen als Grund `OPTION`.

```
try {
    tical::view::month::getData -year 2025
} trap {TICAL VIEW} {msg} {
    puts $msg
}
;# -> view::month::getData: -year and -month are required
```

Die Meldungen stehen im Anhang Fehlermeldungen.

10.13 Tests

`tests/all.tcl` startet jede der 17 Testdateien in einem eigenen Prozess, mit `-encoding utf-8`. Fehlt ein optionales Paket (Tk, `pdf4tcl`, `tdbc`, tclutils), meldet der Lauf die Datei als `SKIP … missing package …`; fehlt ein Paket von tical selbst, ist das ein Fehler; der Exit-Code ist nur bei Fehlern ungleich

1. Tk-Tests brauchen ein Display (`xvfb-run -a tclsh tests/all.tcl`).

`io-import.test` sucht tclutils in `TCLUTILS_TM` (dessen `lib/tm`; leer: nirgends) oder im Geschwisterordner`../tclutils/lib/tm`.

Tcl	Ergebnis
9.0.4 (mit pdf4tcl, tdbc::sqlite3 und tclutils)	17 Dateien, 343 bestanden
8.6.14 (mit pdf4tcl und tclutils, ohne tdbc::sqlite3)	16 Dateien, 309 bestanden; adapter-sqlite.test übersprungen

Unter 8.6.14 mit Latin-1 als Systemkodierung ergibt der Lauf dasselbe; vor der Umstellung auf `source -encoding utf-8` im `pkgIndex.tcl` waren dort zwei Tests rot, und die Monatsnamen kamen als `MÃ¤rz` an.

Ohne `pdf4tcl` überspringt der Lauf auch `render-pdf.test` (19 Tests), ohne tclutils `io-import.test` (23 Tests), `roundtrip.test` (7 Tests: Beispieldateien importieren, exportieren, wieder importieren, Serien auflösen) und drei Vergleiche mit `tuholiday` und `turrule`: ohne beide 285 bestanden unter 9.0.4, 254 unter 8.6.14.

In tkutils 0.49.0 laufen dazu `tkutical.test` (17) und `tkumonthcanvas.test` (13) gegen tical 1.1.1, unter Tcl/Tk 8.6.14 und 9.0.4 ohne Fehler.

Getestet mit tical 1.1.1, Tcl/Tk 8.6.14 und 9.0.4 unter Linux (Xvfb), `pdf4tcl` 0.9.4.73, tclutils 0.69.0; alle Beispiele dieses Kapitels unter beiden Versionen, die SQLite-Beispiele nur unter 9.0.4.

Nicht getestet: Windows, macOS; die Demos unter `demos/` und das Aussehen der PDF-Seiten nur von Hand.

10.14 Merkliste

- tical über `auto_path` laden, nicht über `tcl::tm::path`.
- Kern ohne Tk; Canvas, PDF und SQLite einzeln laden.
- Region bei `-holidays` in beliebiger Schreibung: `DE` oder `de`.
- `-count N` (N > 1) liefert eine Liste von N Monats-ViewSpecs.
- Unbekannte Optionen sind Fehler (`TICAL … OPTION`), auch in Views.

- Alle Renderer beschriften nach `tical::locale` (Vorgabe deutsch); den Wochenstart setzt die View (`-weekStart sun`), nicht der Renderer; `render::term -weekStart` prüft nur.
- Ein Terminformat (`tical::event`): `summary`, `dtstart`, `dtend`; Datum ohne Uhrzeit ist ganztägig, `dtend` dann exklusiv.
- Termine mit `getData … -events` in die View; mehrtägige stehen in jeder berührten Zelle, Serien an jedem ihrer Tage.
- `render::term`: `[17]` heute (`-color 1`), `(18)` Feiertag, `17*` Termin; Woche und Tag als Liste.
- `TZID` wird umgerechnet; unbekannte Zonen bleiben lokale Zeit.
- Oster- und Pfingstsonntag nur mit `holidays::de -sundays 1`.
- Ganztägig wird `VALUE=DATE` geschrieben und gelesen.
- Lesen und expandieren mit `tical::io::import` (braucht tclutils).
- `title`/`date`/`startTime` und `starts_at` gehen in 1.1 noch, in 1.2 nicht mehr.
- Fehler fangen mit `trap {TICAL …}`.

Nächstes Kapitel: Fallen

11 Fallen

- Nur tkutils laden, Provider aus tclutils vergessen.
- ctrlutils 0.3 mit tkudialog 0.2 / altem tablelist-Default.
- `package require ctrlutils` ohne Tk oder ohne Display — es scheitert an `cufileops`. Tk-frei nur `cusync`, `cumonitor`, `cuundo` einzeln laden.
- Schreibmenüs trotz read-only caps.
- Explorer-Code nach Dateiblick kopieren statt Controller reuse.
- App-Workarounds für Multi-Select/Dialog neu erfinden — Libs 0.3/0.4.
- Suite gegen `site-tcl` statt Workspace (Banner lesen). Eine installierte Kopie gleicher Version verdeckt den Baum, wenn sie im Modulpfad vorn steht; nur `tests/all.tcl` schiebt die eigenen Bäume nach vorn.
- Neue Version über eine alte `site-tcl`-Kopie kopiert: `cufileops-0.4.tm` bleibt neben `cufileops-0.5.tm` liegen, ebenso `cuundo-0.3.tm`, `cusync-0.1.tm` und die alten tkutils- und tclutils-Dateien (Liste in Pairing und Setup).
- `setup.tcl` gesourct, ctrlutils-`lib/tm` vergessen — `can't find package ctrlutils`.
- `cufileops::install` ohne `-onop`: Undo bleibt leer, ohne Meldung.
- cufileops und cuundo mit verschiedenem `$f` installiert: Undo findet seine Sätze nicht und frischt die Liste nicht auf.
- `cuundo::install` ohne `-list`: kein Strg+Z.
- `cufilter::widget` aufgerufen, aber nicht gepackt — die Zeile existiert, ist aber unsichtbar.
- Nach eigenen Dateioperationen `cumonitor::poke` vergessen: der Monitor meldet die eigene Änderung als fremde.
- `cumonitor` unter `tclsh` ohne Ereignisschleife (`vwait`): es pollt nie.
- `cumonitor::stop` oder `poke` aufräumend aufgerufen, bevor im Prozess je ein Monitor installiert war: `can't read "state": no such variable`.
- `cusync` für eine Löschsynchronisation halten — es löscht auf dem Ziel nie.
- Eigene App ohne `owSetApp`: ihr "Open with"-Register landet im Konfigurationsordner `explorer`.

- `::ctrlutils::cufileops::owApp` direkt setzen statt `owSetApp` aufrufen: nach dem ersten Zugriff bleibt das alte Register im Speicher.
- Utils-Bäume in Buch-PRs patchen.

11.1 tical

- tical mit `tcl::tm::path add` einhängen: Tcl findet nur das Sammelpaket, dann `can't find package tical::util 1.0`. tical gehört in den `auto_path`.
- `month::getData … -count 3` wie einen einzelnen ViewSpec behandeln: Ab N > 1 kommt eine Liste von Monaten zurück.
- `render::term::print … -weekStart sun` bei einer Montags-View: Fehler `TICAL RENDER WEEKSTART`. Die Option prüft nur; den Wochenstart setzt die View (`getData … -weekStart sun`), die Renderer folgen ihr.
- Tippfehler in Optionen (`-weekstart`, `-weeknumbers`): seit 1.1 ein Fehler `TICAL … OPTION`, vorher still übergangen.
- Ganztägigen Termin mit `dtend` = letzter Tag angeben: `dtend` ist exklusiv, `dtstart 20251020 dtend 20251023` sind drei Tage, nicht vier. Ein `dtend` gleich `dtstart` macht `normalize` zu einem Tag.
- Mehrtägigen Termin nur in der Startzelle suchen: die View legt ihn in jede berührte Zelle, mit `part start`, `middle`, `end`.
- `render::term`-Ausgabe von 1.0.1 erwarten: Zellen sind jetzt vier Zeichen breit, Feiertag `(3)` statt `+3`, heute `[17]` statt `*17`, `*` hinter der Zahl heißt Termin.
- Auf deutsche Fehlertexte prüfen: tical meldet englisch; auf den Errorcode `TICAL …` prüfen, nicht auf den Wortlaut.
- `render::canvas::setCallback {w date} {…}` mit Parameterliste und Rumpf: Fehler `TICAL RENDER ARGS`; ein Kommandopräfix übergeben.
- Auf `starts_at`/`ends_at` oder `title`/`date`/`startTime` bauen: 1.1 nimmt und liefert sie noch, 1.2 nicht mehr; `dtstart`, `dtend`, `summary` nehmen.
- `tical::io::import` ohne tclutils laden: `can't find package tclutils::tuical 0.1`; tclutils in den Modulpfad.
- Auf die Umrechnung von `TZID` bauen, wenn die Datei eigene `VTIMEZONE`-Namen hat: eine Zone, die Tcl nicht kennt, bleibt lokale Zeit; `tical::event::tzid name` liefert dann `""`.
- Importierte Termine mit `tzid` in `tical::adapter::sqlite` speichern: die Tabelle hat keine Spalte dafür, die Zeit gilt danach als lokal. Vorher in UTC umrechnen oder die Zone selbst ablegen.
- `tuholiday` und `tical::holidays::de` gleichsetzen: `holidays::de` lässt Oster- und Pfingstsonntag weg; `getHolidays -sundays 1` gibt dieselbe Liste wie `tuholiday`.

11.2 Behoben in ctrlutils 0.4

Diese Fallen gelten für ctrlutils 0.3. In ctrlutils 0.4 sind sie behoben:

- `tkutils::tkufilelist` nicht geladen, weil `ctrlutils` es mitzubringen scheint — in 0.3 tut es das nicht; bis 0.3 nötig: selbst laden. Jetzt lädt `cufileops` es selbst.
- "Open with": in 0.3 teilen sich alle Apps das Register `explorer`, und `owApp` nach dem ersten Zugriff zu setzen wirkt nicht. Jetzt `owSetApp`.
- Fehlerdialoge von cuundo ohne `-parent`: in 0.3 können sie hinter dem App-Fenster aufgehen. Jetzt über dem Toplevel der Liste.

- Gründe von cuundo und die Filterzeile von cufilter: in 0.3 immer englisch. Jetzt über msgcat.
- Gescheitertes Redo: in 0.3 ist der Satz weg, ein zweiter Versuch geht nicht. Jetzt bleibt er auf dem Redo-Stapel.
- Tippfehler in einer Option von `cusync::apply`: in 0.3 still übergangen. Jetzt ein Fehler `unknown option "…" mit Known: -onfile -ondir`.
- `cusync` mit ZIP-, DAV- oder SFTP-Ziel: in 0.3 kann eine gleichnamige lokale Datei die mtime der Quelle bekommen. Jetzt nur noch zwischen zwei lokalen Providern.
- Tests ohne Display: in 0.3 scheitert `smoke.test` in `explorer-0-load`. Jetzt übersprungen.
- Explorer startet nicht: in 0.3 nennt die Meldung nur den `auto_path`, obwohl die Module über den Modulpfad kommen. Jetzt beide.

11.3 Behoben in ctrlutils 0.5

Diese Fallen gelten für ctrlutils 0.4:

- `cusync::compare`: Scheitert das Auflisten einer Seite, gilt sie als leer. Beim Ziel heißt das: Jede Datei der Quelle wird kopiert, auch über neuere Dateien. Jetzt ein Fehler `{CTRLUTILS CUSYNC LIST …}`.
- Cut mit Teilfehler: Nach dem Paste ist der ganze Cut weg, auch für die Einträge, die nicht kopiert wurden. Jetzt bleiben sie in der Zwischenablage.
- Undo frischt die Liste über eine interne Prozedur von cufileops auf und verschluckt einen Fehler dabei. Jetzt `-refresh` und ein Fehler `{CTRLUTILS CUUNDO REFRESH}`.
- `cufilter`: Ein Filter, der sich nicht setzen lässt, verschwindet still. Jetzt `invalid`, `<<CufilterError>>` und `lastError`.
- Alte Moduldateien: Der Baum von ctrlutils 0.4 enthielt noch `cufileops-0.4.tm`, `cuundo-0.3.tm` und `cusync-0.1.tm`. Wer `lib/tm` kopiert, nimmt sie mit; ein `package require -exact … 0.3` lädt sie.

11.4 Zusammenfassung

- Die meisten Fallen sind stille Verdrahtungsfehler: falsches `$f`, fehlendes `-onop`, fehlendes `poke`, nicht gepackte Filterzeile.
- Bei Versions- und Pfadfragen zuerst das Banner der Suite lesen.
- Bei tical: Ladeweg (`auto_path`), Groß-/Kleinschreibung der Region, exklusives `dtend` ganztägiger Termine.

Nächstes Kapitel: Module

12 Module (Stack-Sicht)

Schicht	Buch / Ort	Umfang
tclutils	Buch tclutils, tclutils/docs/	148 Moduldateien
tkutils	Buch tkutils, tkutils/docs/	79 Module
ctrlutils	dieses Buch, ctrlutils/docs/	5 Module
tical	dieses Buch, Kapitel 10, tical/doc/, tical/schemas/	19 Pakete

Gezählt als `.tm`-Dateien unter `lib/tm/<lib>/` ohne das Sammelpaket. tclutils 0.69.0 hat 142

flache Module und vier Provider in `tuprovider/` (`dav`, `ftp`, `sftp`, `zip`). 0.69.0 bringt `tusysinfo` und `tuedid` neu und `tucsv`, `tusqlite` als 0.2. 0.68.0 brachte kein neues Modul, aber neue Nummern für 76 vorhandene, darunter `tuprovider` 0.2, `tuprovider::sftp` 0.2 und `tuopen` 0.2. `tupdftext` kam mit 0.67.0 dazu, `tutask` mit 0.65.0 (seit 0.66.0 als 0.2). In tkutils 0.49.0 neu: `tkutlfilter`, `tkutlgroup`, `tkutldrag`, `tkusysinfo`; neu nummeriert `tkutablelist` 0.5, `tkutlstate` 0.4, `tkutlclip`, `tkutlfind`, `tkutlsort`, `tkutltree`, `tkutltools` 0.3. In 0.48.0 neu nummeriert: `tkulauncher`, `tkuload` 0.3; in 0.47.0 `tkufiledialog` 0.2, `tkutlclip`, `tkutlfind`, `tkutlsort`, `tkutltools`, `tkutltree` 0.2, `tkutlfmt`, `tkutlfooter`, `tkutlstate` 0.3.

Bei tkutils liegt `tkutablelist` nur noch als 0.5 bei; die Dateien für 0.3 und 0.4 sind entfernt.

ctrlutils vollständig:

Modul	Version	braucht Tk	Kernaufrufe
cufileops	0.6	ja	install … ?-onop?, newFolder, openFile, openWith, openWithDefault, rename, copy, cut, paste, delete, refresh
cuundo	0.5	nur -list	install … ?-list? ?-refresh?, record, undo, redo, canUndo, canRedo, clear
cufilter	0.2	ja	widget $path $list, clear, lastError
cumonitor	0.2	nein	install … ?-interval? ?-onchange?, watch, poke, stop
cusync	0.3	nein	compare, plan, apply, applyOne

Dateien: `lib/tm/ctrlutils-0.5.tm`, in `lib/tm/ctrlutils/` `cufileops-0.6.tm`, `cufilter-0.2.tm`, `cumonitor-0.2.tm`, `cusync-0.3.tm`, `cuundo-0.5.tm`.

Dazu für das "Open with"-Register in cufileops: `owSetApp` (App-Name, seit 0.5), `owApps`, `owDefault`, `owAddApp`, `owRemoveApp`, `owSetDefault` (Schlüssel: Endung in Kleinbuchstaben mit Punkt, z. B. `.txt`).

Sammelpaket: `ctrlutils` 0.5 lädt alle fünf.

Versionsgeschichte: ctrlutils 0.3 enthielt zuerst `cufileops` 0.3 und `cuundo` 0.2, nach einem Nachtrag `cufileops` 0.4 und `cuundo` 0.3, dazu `cusync` 0.1. Ab 0.4 steigt das Sammelpaket mit jedem Modul.

12.1 tical 1.1.1

Alle Pakete stehen in `tical/pkgIndex.tcl`. Was sich in 1.1.1 geändert hat, trägt 1.1.1 (`tical::holidays::de` 1.0.1; `tical::config`, `tical::locale` und `tical::holidays` 1.0.2); `tical::term` bleibt 1.0.

Paket	Datei	braucht	im Sammelpaket
tical	tical-1.1.1.tm	—	(Sammelpaket)
tical::core	core/tical-core-1.1.1.tm	—	ja
tical::util	core/tical-util-1.1.1.tm	—	ja

Paket	Datei	braucht	im Sammelpaket
tical::config	core/tical-config-1.0.2.tm	—	ja
tical::locale	core/tical-locale-1.0.2.tm	—	ja
tical::term	core/tical-term-1.0.tm	—	ja
tical::holidays	core/tical-holidays-1.0.2.tm	—	ja
tical::holidays::de	holidays/tical-holidays-de-1.0.1.tm	—	ja
tical::event	core/tical-event-1.1.1.tm	—	ja
tical::view::month	views/tical-view-month-1.1.1.tm	—	ja
tical::view::week	views/tical-view-week-1.1.1.tm	—	ja
tical::view::day	views/tical-view-day-1.1.1.tm	—	ja
tical::render::term	render/tical-render-term-1.1.1.tm	—	ja
tical::render::canvas	render/tical-render-canvas-1.1.1.tm	Tk	nein
tical::render::pdf	render/tical-render-pdf-1.1.1.tm	pdf4tcl	nein
tical::model	model/tical-model-1.1.1.tm	—	ja
tical::io::ical	io/tical-io-ical-1.1.1.tm	—	ja
tical::io::import	io/tical-io-import-1.1.1.tm	tclutils (tuical, turrule)	nein
tical::adapter::sqlite	adapter/tical-adapter-sqlite-1.1.1.tm	tdbc::sqlite3	nein

Auf tical bauen in tkutils 0.49.0 `tkutils::tkutical` 0.4 und `tkutils::tkumonthcanvas` 0.5; beide fehlen im Sammelpaket `tkutils`.

12.2 Zusammenfassung

- tclutils 148 Moduldateien, tkutils 79, ctrlutils 5 Module, tical 19 Pakete.
- ctrlutils 0.5 enthält cufileops 0.6, cuundo 0.5, cusync 0.3 und cufilter 0.2.
- Das Sammelpaket `tical` lädt alles außer Canvas, PDF, SQLite und Import.

Nächstes Kapitel: Glossar

13 Glossar

Pairing
Abgestimmte Versionsnummern der drei Libs; derzeit tclutils 0.69.0, tkutils 0.49.0, ctrlutils 0.5.

Provider
Einheitliche Speicher-API (tclutils); Schemata `local`, `zip`, `dav`, `ftp`, `sftp`.

Controller
ctrlutils-Modul: verdrahtet Verhalten auf vorhandenen Widgets. Ausnahmen: `cufilter` baut eine eigene Filterzeile, `cusync` braucht kein Tk.

caps
Provider-Fähigkeiten; steuern Menüs. Beispiel ZIP: `get list stat`.

Operationssatz
Dict, das cufileops nach jeder erfolgreichen Operation an `-onop` gibt (`op` `mkdir|rename|paste|delete|movex` plus Pfade); cuundo zeichnet es auf.

Schnappschuss
Inhalt, den cufileops vor dem Löschen festhält (Bytes oder Teilbaum), damit cuundo ihn wiederherstellen kann.

Polling
Regelmäßiges `list` über den Provider (cumonitor); funktioniert lokal, im ZIP und entfernt gleich.

Sync-Plan
Ergebnis von `cusync::plan`: Kopieraktionen für eine Richtung, nie Löschungen.

msgcat
Tcl-Paket für Übersetzungen; in ctrlutils 0.4 holen cufileops, cuundo und cufilter ihre Texte daraus, Katalog im Namensraum `::ctrlutils`.

Testgrenze
Zeitlimit je Testdatei in Sekunden (CTRLUTILS_TESTGRENZE usw., Standard 120).

site-tcl
Verzeichnis im Standard-Modulpfad einer Tcl-Installation (`lib/tcl8/site-tcl`, `lib/tcl9/site-tcl`).

Explorer
Referenz-App in ctrlutils.

Dateiblick
Endbenutzer-App auf dem Stack.

tical
Kalender- und iCalendar-Engine (1.1) in eigenem Repo; Kern ohne Tk, Renderer für Text, Tk-Canvas und PDF; geladen über `auto_path`.

tical::event
Terminformat von tical (`summary`, `dtstart`, `dtend`, `allday` …) mit `normalize` und `validate`; gilt für Views, Modell, iCalendar und SQLite.

ViewSpec
Dict einer tical-View (`month-grid`, `week-grid`, `day-grid`) mit einem Raster aus Zellen; Eingabe jedes Renderers.

Feiertags-Plugin
Befehl, der für ein Jahr ein Dict `Datum → Name` liefert, unter einer Region bei

`tical::holidays` registriert (DE).

13.1 Zusammenfassung

- Die Begriffe oben tragen das Buch: Pairing, Provider, caps, Operationssatz, Testgrenze, ViewSpec.

Nächstes Kapitel: Fehlermeldungen

14 Fehlermeldungen

Wortlaut aus dem Code. Die mit "geprüft" markierten Zeilen sind mit Tcl/Tk 8.6.14 unter Linux (Xvfb) ausgelöst; die Zeilen zu den Optionen von cufileops, cuundo, cumonitor und cusync, zu `owSetApp`, zum Explorer-Start und zu tical auch mit Tcl/Tk 9.0.4. Wortlaut gilt für ctrlutils 0.5 und tical 1.1.1; wo ein älteres ctrlutils anders ist, steht es dabei.

Symptom	Prüfen
can't find package ctrlutils /ctrlutils::... (geprüft)	tcl::tm::path add .../ctrlutils/lib/tm; setup.tcl kennt ctrlutils nicht
can't find package tkutils::tkudialog 0.3 (geprüft)	tkutils ≥ 0.46 (Dialog 0.3)
no display name and no $DISPLAY environment variable beim Laden von ctrlutils (geprüft)	Sammelpaket braucht Tk; ohne Display nur Tk-freie Controller laden
invalid command name "::tkutils::tkufilelist::widget" (geprüft)	ctrlutils 0.3: package require tkutils::tkufilelist fehlt; ab ctrlutils 0.4 lädt cufileops es selbst
Dialog ohne Default-Button	tkudialog ≥ 0.3
Multi-Select weg nach Rechtsklick	cufileops ≥ 0.3 + tablelist 0.4
can't find package tclutils::tuprovider	tclutils laden
Suite testet falschen Baum	Banner /TCL*_TM_PATH; Suite über tests/all.tcl starten
unknown option "-onpo" /Known: -onop (geprüft)	Tippfehler; Errorcode CTRLUTILS CUFILEOPS OPTION -onpo
missing value for option "-onop" (geprüft)	Option ohne Wert; Errorcode CTRLUTILS CUFILEOPS OPTION
unknown option "..." /Known: -list, -refresh (geprüft)	cuundo::install; Errorcode CTRLUTILS CUUNDO OPTION ...; bis ctrlutils 0.4 nur Known: -list
list not refreshed: ... (geprüft)	Undo oder Redo ist getan, das Auffrischen der Liste nicht; Errorcode CTRLUTILS CUUNDO REFRESH; Skript von -refresh oder von cufileops::install prüfen

Symptom	Prüfen
unknown option "..." /Known: -interval -onchange (geprüft)	cumonitor::install; Errorcode CTRLUTILS CUMONITOR OPTION ...
cumonitor: <f> not installed (geprüft)	watch vor cumonitor::install
can't read "state": no such variable (geprüft)	cumonitor::watch, poke oder stop, bevor im Prozess ein Monitor installiert war
unknown option "-x" /Known: -onfile -ondir (geprüft)	cusync::apply; Errorcode CTRLUTILS CUSYNC OPTION -x; in ctrlutils 0.3 still übergangen
missing value for option "-onfile" (geprüft)	cusync::apply, Option ohne Wert; Errorcode CTRLUTILS CUSYNC OPTION
cannot list "..." (R): ... (geprüft)	cusync::compare: ein Verzeichnis lässt sich nicht auflisten; Errorcode CTRLUTILS CUSYNC LIST R dir (L für links); bis ctrlutils 0.4 galt die Seite als leer
invalid application name "mein app": use letters, digits, . _ -(geprüft)	cufileops::owSetApp; nur Buchstaben, Ziffern, . _ -; Errorcode CTRLUTILS CUFILEOPS APPNAME {mein app}
"Open with" zeigt die Programme einer anderen App	kein cufileops::owSetApp gesetzt (Register explorer) oder in ctrlutils 0.3 owApp erst nach dem ersten Zugriff gesetzt
Undo tut nichts	cufileops ohne -onop [list ::ctrlutils::cuundo::record $f] oder anderes $f
Strg+Z ohne Wirkung	cuundo::install ohne -list
Undo failed: + no saved content to restore (geprüft)	Delete ohne Schnappschuss (Provider konnte nicht lesen)
Redo failed: (geprüft)	Ziel blockiert; der Satz bleibt auf dem Redo-Stapel (in ctrlutils 0.3 ist er danach verloren)
Cut is not available here (the source cannot delete). (geprüft)	Quelle ohne delete, z. B. ZIP; Copy nehmen
Delete is not available here (the source cannot delete). (geprüft)	Quelle ohne delete
Copied, but could not remove the source:	Cut: Kopie steht, Quelle blieb
Filterzeile fehlt	cufilter::widget erzeugt, aber nicht gepackt
Monitor meldet eigene Änderungen	nach der Operation cumonitor::poke
Menü zeigt nur "(read-only)" (geprüft)	keine passende Operation, z. B. ZIP ohne markierte Datei und mit leerer Zwischenablage

Symptom	Prüfen
Explorer: "Explorer cannot start", Missing package:, Searched module path (tcl::tm::path):, Searched auto_path: (geprüft)	Modulpfad für alle drei Libs; Tablelist. In ctrlutils 0.3 ohne die Zeilen zum Modulpfad
Explorer: Sync needs the ctrlutils::cusync module.	Modul cusync fehlt
Explorer: Find in list needs the tkutils::tkutlfind module.	Modul tkutlfind fehlt
all.tcl: tclutils::tutestrun not found (set TCLUTILS_TM) (geprüft, Exit 2)	tclutils ≥ 0.64 als Geschwister oder TCLUTILS_TM setzen
<datei> (hang after 120s)	CTRLUTILS_TESTGRENZE erhöhen
smoke.test (1 failed), couldn't connect to display (geprüft)	ctrlutils 0.3: Suite mit Display starten (xvfb-run -a); ab ctrlutils 0.4 übersprungen
can't find package tical::util 1.0 (geprüft)	tical über tcl::tm::path eingehängt; tical-Ordner in den auto_path
can't find package tical::view::month o. ä.	tical fehlt im auto_path; für tkutils TICAL_DIR setzen oder tical als Geschwister-Checkout
Kalender zeigt keine Feiertage	unbekannte Region bei -holidays (keine Fehlermeldung)
view::month::getData: -year and -month are required (geprüft)	Monat ohne -month; -count ersetzt es nicht; Errorcode TICAL VIEW ARGS
view::month::getData: -count must be an integer >= 1, got "0" (geprüft)	-count kleiner 1 oder keine Zahl; Errorcode TICAL VIEW COUNT
view::month::getData: unknown option "-weekstart", must be -count, -events, ... (geprüft)	Tippfehler in einer Option der View (bis 1.0.1 still übergangen); Errorcode TICAL VIEW OPTION
render::term::print: unknown option "-colour", must be -color, -weekNumbers or -weekStart (geprüft)	Option, die render::term nicht kennt; Errorcode TICAL RENDER OPTION
render::term::print: -weekStart mon contradicts the view (starts on sun): week start comes from the view; use getData ... -weekStart mon (geprüft)	-weekStart passt nicht zur View; den Wochenstart in getData setzen; Errorcode TICAL RENDER WEEKSTART
render::pdf::create: bad paper "b5": must be a3, a4, a5 or letter (geprüft)	Papierformat, das render::pdf nicht kennt; Errorcode TICAL RENDER OPTION
event missing required field: dtstart (geprüft)	Termin ohne Beginn bei getData ... -events, normalize, model oder exportEvents; Errorcode TICAL EVENT FIELD

Symptom	Prüfen
dtend "20251014T090000" is before dtstart "20251014T100000" (geprüft)	Ende vor Beginn; Errorcode TICAL EVENT RANGE
render::term::print: expected type month-grid, week-grid or day-grid, got "year-grid" (geprüft)	ViewSpec, den render::term nicht kennt (bis 1.1 auch Woche und Tag); Errorcode TICAL RENDER TYPE
setCallback: "w date" is not a widget path; pass one command prefix (setCallback CMD) or a canvas and one (setCallback .c CMD) (geprüft)	setCallback mit Parameterliste und Rumpf; ein Kommandopräfix übergeben; Errorcode TICAL RENDER ARGS
dtstart format invalid: "2025-10-14" (expected YYYYMMDD, YYYYMMDDTHHMMSS or YYYYMMDDTHHMMSSZ) (geprüft)	io::ical::validate mit ISO-Datum; validate prüft die Form, in der geschrieben wird; Errorcode TICAL ICAL FORMAT
FREQ must be DAILY|WEEKLY|MONTHLY|YEARLY, got "HOURLY" (geprüft)	rrule mit anderer Frequenz; Errorcode TICAL ICAL RRULE
appointment not found: appt-9 (geprüft)	tical::model::get... mit unbekannter Kennung; Errorcode TICAL MODEL NOTFOUND
event not found: nix (geprüft, nur 9.0.4)	adapter::sqlite::read/update/delete mit unbekannter uid; Errorcode TICAL SQLITE NOTFOUND
can't find package tdbc::sqlite3	tical::adapter::sqlite ohne TDBC-SQLite; die Suite überspringt adapter-sqlite.test
can't find package tclutils::tuical 0.1 (geprüft)	tical::io::import ohne tclutils; tcl::tm::path add …/tclutils/lib/tm; die Suite überspringt io-import.test
VEVENT "X": RRULE FREQ=MONTHLY;BYSETPOS=-1: unsupported rule part: BYSETPOS (geprüft)	io::import::expand mit einem RRULE-Teil, den turrule nicht kennt; Errorcode TICAL IMPORT RRULE

Fehlercodes der Einzelmodule: jeweilige Lib-Docs und Bücher *tclutils* und *tkutils*.

14.1 Zusammenfassung

- Ladefehler: Modulpfad, Tk/Display, `tkufilelist` selbst laden.
- Optionsfehler nennen die bekannten Optionen und tragen einen Errorcode `CTRLUTILS <MODUL> OPTION …`.
- Testfehler: Banner, Display, `*_TESTGRENZE`.
- tical: `auto_path` statt Modulpfad; der Import braucht tclutils im Modulpfad.

15 Lizenz

Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Namensnennung - Weitergabe unter gleichen
Bedingungen 4.0 International (CC BY-SA 4.0)
Codebeispiele des Autors: MIT-Lizenz (unten)

Text

Dieses Buch ist unter der Creative-Commons-Lizenz
"Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0) lizenziert.

Sie dürfen das Material in jedwedem Format oder Medium vervielfältigen
und weiterverbreiten sowie es remixen, verändern und darauf aufbauen,
auch zu kommerziellen Zwecken, unter den folgenden Bedingungen:

* Namensnennung: Sie müssen angemessene Urheber- und Rechteangaben
  machen, einen Link zur Lizenz beifügen und angeben, ob Änderungen
  vorgenommen wurden.

* Weitergabe unter gleichen Bedingungen: Wenn Sie das Material
  remixen, verändern oder anderweitig direkt darauf aufbauen, dürfen
  Sie Ihre Beiträge nur unter derselben Lizenz wie das Original
  verbreiten.

Verbindlich ist der vollständige Lizenztext:
https://creativecommons.org/licenses/by-sa/4.0/legalcode.de
Englische Originalfassung:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

Codebeispiele

Die Codebeispiele des Autors (in den Codeblöcken des Buchs und als
Skripte daneben) stehen zusätzlich unter der MIT-Lizenz. Übernommene
Codebeispiele und zitierte Texte aus anderen Projekten (Tcl/Tk,
pdf4tcl und andere) behalten die Lizenz ihres Originals; die Quelle ist
jeweils angegeben.

MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be

```
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Stichwortverzeichnis

C

caps 17, 45
Controller 12, 45
ctrlutils 4
CTRLUTILS_TESTGRENZE 10
cufileops 17
cufilter 13
cumonitor 14
cusync 15
cuundo 20

D

Dateiblick 45

E

Explorer 45

F

Feiertags-Plugin 45

M

Mehrfachauswahl 19
Modulpfad 6, 26
msgcat 20, 45

O

Operationssatz 21, 45

P

Pairing 45
Polling 45
Provider 45

S

Schnappschuss 45
site-tcl 7, 45
Sync-Plan 45

T

tclutils 4
Testgrenze 45
tical 4, 25, 45
tical::event 29, 45
tical::io::import 35
tical::locale 31
tkutils 4
tuholiday 27
tuical 27
turrule 27

tutestrun 9

V

ViewSpec 25, 45